

数据的机器级表示与处理

在高级语言程序中需要定义所处理数据的类型以及存储的数据结构。例如，C 语言程序中有无符号整数类型（unsigned int）、带符号整数类型（int）、单精度浮点数类型（float）等；此外，在 C 语言中，多个相同类型数据可以构成一个数组（array），多个不同类型数据可以构成结构（struct）。那么，在高级语言程序中定义的这些数据在计算机内部是如何表示的？它们在计算机中又是如何存储、运算和传送的呢？

本章重点讨论数据在计算机内部的机器级表示和基本处理方式。主要内容包括：进位计数制、二进制定点数的编码表示、无符号整数和带符号整数的表示、IEEE 754 浮点数表示标准、西文字符和汉字的编码表示、十进制数的二进制编码表示（即 BCD 码）、C 语言中各种类型数据的表示和转换、数据的宽度和存放顺序、基本运算及其运算电路。

2.1 数制和编码

2.1.1 信息的二进制编码

数据是计算机处理的对象。从不同的处理角度来看，数据有不同的表现形态。从外部形式来看，计算机可处理数值、文字、图、声音、视频以及各种模拟信息，它们被称为感觉媒体。从算法描述的角度来看，有图、表、树、队列、矩阵等结构类型的数据。从高级语言程序员的角度来看，有数组、结构、指针、实数、整数、布尔数、字符和字符串等类型的数据。不管以什么形态出现，在计算机内部数据最终都由机器指令来处理。而从机器指令的角度来看，数据只有整数、浮点数和位串这几类简单的基本数据类型。

计算机内部处理的所有数据都必须是“数字化编码”了的数据。现实世界中的感觉媒体信息由输入设备转化为二进制编码表示，因此，输入设备必须具有“离散化”和“编码”两方面的功能。因为计算机中用来存储、加工和传输数据的部件都是位数有限的部件，所以，计算机中只能表示和处理离散的信息。数字化编码过程，就是指对感觉媒体信息进行采样，将现实世界中的连续信息转换为计算机中的离散的“样本”信息，然后对样本信息用“0”和“1”进行数字化编码的过程。所谓编码，就是用少量简单的基本符号，对大量复杂多样的信息进行一定规律的组合。基本符号的种类和组合规则是信息编码的两大要素。例如，电报码中用 4 位十进制数字表示汉字，从键盘上输入汉字时用汉语拼音（即 26 个英文字母）表示汉字等，都

是编码的典型例子。

在计算机系统内部，所有信息都是用二进制进行编码的。也就是说计算机内部采用的是二进制表示方式。这样做的原因有以下几点。

① 二进制只有两种基本状态，使用有两个稳定状态的物理器件就可以表示二进制数的每一位，而制造有两个稳定状态的物理器件要比制造有多个稳定状态的物理器件容易得多。例如，用高、低两个电位，或用脉冲的有无、脉冲的正负极性等都可以很方便、很可靠地表示“0”和“1”。

② 二进制的编码、计数和运算规则都很简单，可用开关电路实现，简便易行。

③ 两个符号“1”和“0”正好与逻辑命题的两个值“真”和“假”相对应，为计算机中实现逻辑运算和程序中的逻辑判断提供了便利的条件，特别是能通过逻辑门电路方便地实现算术运算。

采用二进制编码将各种媒体信息转变成数字化信息后，可以在计算机内部进行存储、运算和传送。在高级语言程序中，可以用利用图、树、表和队列等数据结构进行算法描述，并以数组、结构、指针和字符串等数据类型来说明处理对象，但将高级语言程序转换为机器语言程序后，每条机器指令的操作数就只能是以下4种简单的基本数据类型：无符号定点整数、带符号定点整数、浮点数和非数值型数据（位串），如图2.1中虚线框内所示。

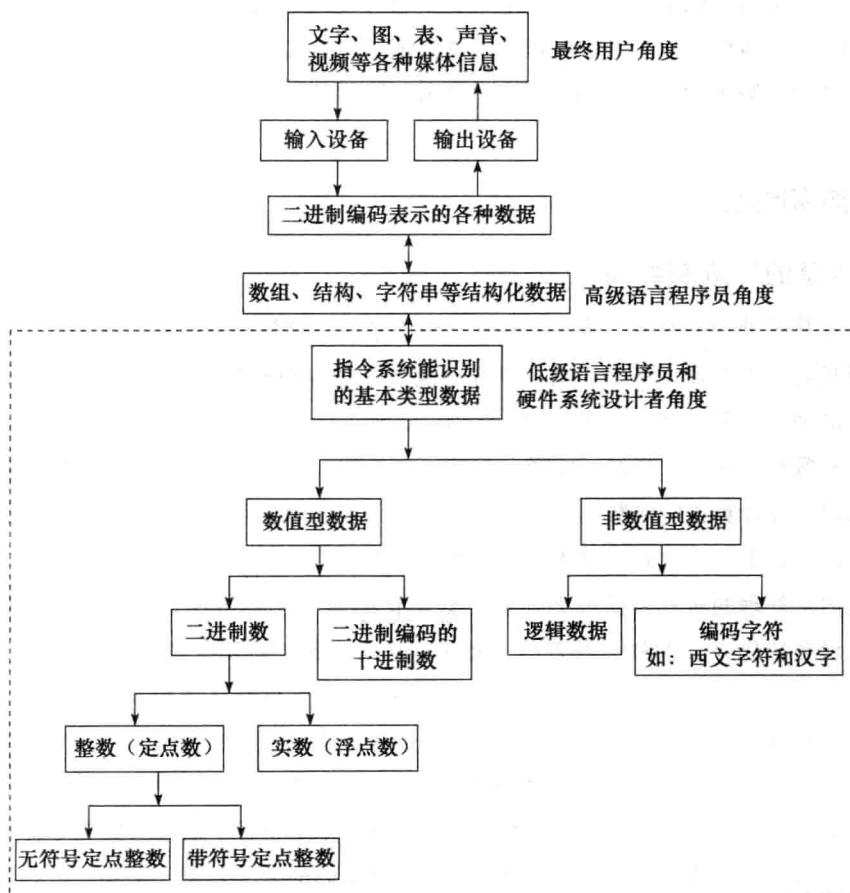


图 2.1 计算机外部信息与内部数据的转换

指令所处理的数据类型分为数值数据和非数值数据两种。数值数据可用来表示数量的多少,可比较其大小,分为整数和实数,整数又分为无符号整数和带符号整数。在计算机内部,整数用定点数表示,实数用浮点数表示。非数值数据就是一个没有大小之分的位串,不表示数量的多少,主要用来表示字符数据和逻辑数据。

日常生活中,常使用带正负号的十进制数表示数值数据,例如 6.18、-127 等。但这种形式的数据在计算机内部难以直接存储、运算和传送,仅用来作为程序的输入或输出形式,以方便用户从键盘等输入设备输入数据,或从屏幕、打印机等输出设备上输出数据,它不是用于计算机内部运算和传输的主要表示形式。

在计算机内部,数值数据的表示方法有两大类:第一种是直接用二进制数表示;另一种是采用二进制编码的十进制数 (Binary Coded Decimal Number, 简称BCD) 表示。

表示一个数值数据要确定三个要素:进位计数制、定/浮点表示和编码规则。任何给定的一个二进制 0/1 序列,在未确定它采用什么进位计数制、定点还是浮点表示以及编码表示方法之前,它所代表的数值数据的值是无法确定的。

2.1.2 进位计数制

日常生活中基本上都使用十进制数,其每个数位可用 10 个不同符号 0,1,2,...,9 来表示,每个符号处在十进制数中不同位置时,所代表的数值不一样。例如,2585.62 代表的值是:

$$(2585.62)_{10} = 2 \times 10^3 + 5 \times 10^2 + 8 \times 10^1 + 5 \times 10^0 + 6 \times 10^{-1} + 2 \times 10^{-2}$$

一般地,任意一个十进制数

$$D = d_n d_{n-1} \cdots d_1 d_0 . d_{-1} d_{-2} \cdots d_{-m} \quad (m, n \text{ 为正整数})$$

其值可表示为如下形式:

$$V(D) = d_n \times 10^n + d_{n-1} \times 10^{n-1} + \cdots + d_1 \times 10^1 + d_0 \times 10^0 \\ + d_{-1} \times 10^{-1} + d_{-2} \times 10^{-2} + \cdots + d_{-m} \times 10^{-m}$$

其中 $d_i (i = n, n-1, \cdots, 1, 0, -1, -2, \cdots, -m)$ 可以是 0,1,2,3,4,5,6,7,8,9 这 10 个数字符号中的任何一个,“10”称为基数 (base),它代表每个数位上可以使用的不同数字符号个数。 10^i 称为第 i 位上的权。在十进制数进行运算时,每位计满十之后就要向高位进一,即日常所说的“逢十进一”。

类似地,二进制数的基数是 2,只使用两个不同的数字符号 0 和 1,运算时采用“逢二进一”的规则,第 i 位上的权是 2^i 。例如,二进制数 $(100101.01)_2$ 代表的值是:

$$(100101.01)_2 = 1 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} \\ = (37.25)_{10}$$

一般地,任意一个二进制数

$$B = b_n b_{n-1} \cdots b_1 b_0 . b_{-1} b_{-2} \cdots b_{-m} \quad (m, n \text{ 为正整数})$$

其值可表示为如下形式:

$$V(B) = b_n \times 2^n + b_{n-1} \times 2^{n-1} + \cdots + b_1 \times 2^1 + b_0 \times 2^0 \\ + b_{-1} \times 2^{-1} + b_{-2} \times 2^{-2} + \cdots + b_{-m} \times 2^{-m}$$

其中 $b_i (i = n, n-1, \dots, 1, 0, -1, -2, \dots, -m)$ 只可以是 0 和 1 两种不同的数字符号。

扩展到一般情况, 在 R 进制数字系统中, 应采用 R 个基本符号 $(0, 1, 2, \dots, R-1)$ 表示各位上的数字, 采用“逢 R 进一”的运算规则, 对于每一个数位 i , 该位上的权为 R^i 。 R 被称为该数字系统的基。

在计算机系统中使用的常用进位计数制有下列几种。

二进制 $R=2$, 基本符号为 0 和 1。

八进制 $R=8$, 基本符号为 0, 1, 2, 3, 4, 5, 6, 7。

十进制 $R=10$, 基本符号为 0, 1, 2, 3, 4, 5, 6, 7, 8, 9。

十六进制 $R=16$, 基本符号为 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F。

表 2.1 列出了二、八、十、十六进制 4 种进位计数制中各基本数之间的对应关系。

表 2.1 4 种进位计数制之间的对应关系

二进制数	八进制数	十进制数	十六进制数	二进制数	八进制数	十进制数	十六进制数
0000	0	0	0	1000	10	8	8
0001	1	1	1	1001	11	9	9
0010	2	2	2	1010	12	10	A
0011	3	3	3	1011	13	11	B
0100	4	4	4	1100	14	12	C
0101	5	5	5	1101	15	13	D
0110	6	6	6	1110	16	14	E
0111	7	7	7	1111	17	15	F

从表 2.1 中可看出, 十六进制的前 10 个数字与十进制中前 10 个数字相同, 后 6 个基本符号 A, B, C, D, E, F 的值分别为十进制的 10, 11, 12, 13, 14, 15。在书写时可使用后缀字母标识该数的进位计数制, 一般用 B(Binary) 表示二进制, 用 O(Octal) 表示八进制, 用 D(Decimal) 表示十进制 (十进制数的后缀可以省略), 而 H(Hexadecimal) 则是十六进制数的后缀, 有时也在一个十六进制数之前用 0x 作为前缀, 例如二进制数 10011B, 十进制数 56D 或 56, 十六进制数 308FH 或 0x308F 等。

计算机内部所有的信息采用二进制编码表示。但在计算机外部, 为了书写和阅读的方便, 大都采用八、十或十六进制表示形式。因此, 计算机在数据输入后或输出前都必须实现这些进位制数和二进制数之间的转换。以下介绍各进位计数制之间数据的转换方法。

1. R 进制数转换成十进制数

任何一个 R 进制数转换成十进制数时, 只要“按权展开”即可。

例 2.1 将二进制数 $(10101.01)_2$ 转换成十进制数。

解 $(10101.01)_2 = (1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2})_{10} = (21.25)_{10}$ ■

例 2.2 将八进制数 $(307.6)_8$ 转换成十进制数。

解 $(307.6)_8 = (3 \times 8^2 + 7 \times 8^0 + 6 \times 8^{-1})_{10} = (199.75)_{10}$ ■

例 2.3 将十六进制数 $(3A.C)_{16}$ 转换成十进制数。

解 $(3A.C)_{16} = (3 \times 16^1 + 10 \times 16^0 + 12 \times 16^{-1})_{10} = (58.75)_{10}$ ■

2. 十进制数转换成 R 进制数

任何一个十进制数转换成 R 进制数时, 要将整数和小数部分分别进行转换。

(1) 整数部分的转换

整数部分的转换方法是“除基取余，上右下左”。也就是说，用要转换的十进制整数去除以基数 R ，将得到的余数作为结果数据中各位的数字，直到余数为 0 为止。上面的余数（先得到的余数）作为右边低位上的数位，下面的余数作为左边高位上的数位。

例 2.4 将十进制整数 135 分别转换成八进制数和二进制数。

解 将 135 分别除以 8 和 2，将每次的余数按从低位到高位顺序排列如下：

	余数	低位
8 135	7	↑ 高位
8 16	0	
8 2	2	
0		

	余数	低位
2 135	1	↑ 高位
2 67	1	
2 33	1	
2 16	0	
2 8	0	
2 4	0	
2 2	0	
2 1	1	
0		

所以， $(135)_{10} = (207)_8 = (10000111)_2$ 。

(2) 小数部分的转换

小数部分的转换方法是“乘基取整，上左下右”。也就是说，用要转换的十进制小数去乘以基数 R ，将得到的乘积的整数部分作为结果数据中各位的数字，小数部分继续与基数 R 相乘。以此类推，直到某一步乘积的小数部分为 0 或已得到希望的位数为止。最后，将上面的整数部分作为左边高位上的数位，下面的整数部分作为右边低位上的数位。

例 2.5 将十进制小数 0.6875 分别转换成二进制数和八进制数。

解 $0.6875 \times 2 = 1.375$	整数部分 = 1	(高位)
$0.375 \times 2 = 0.75$	整数部分 = 0	↓
$0.75 \times 2 = 1.5$	整数部分 = 1	↓
$0.5 \times 2 = 1.0$	整数部分 = 1	(低位)

所以， $(0.6875)_{10} = (0.1011)_2$ 。

$0.6875 \times 8 = 5.5$	整数部分 = 5	(高位)
$0.5 \times 8 = 4.0$	整数部分 = 4	(低位)

所以， $(0.6875)_{10} = (0.54)_8$ 。

在转换过程中，可能乘积的小数部分总得不到 0，即转换得到希望的位数后还有余数，这种情况下得到的是近似值。

例 2.6 将十进制小数 0.63 转换成二进制数。

解 $0.63 \times 2 = 1.26$	整数部分 = 1	(高位)
$0.26 \times 2 = 0.52$	整数部分 = 0	↓
$0.52 \times 2 = 1.04$	整数部分 = 1	↓

$$0.04 \times 2 = 0.08 \quad \text{整数部分} = 0 \quad (\text{低位})$$

所以, $(0.63)_{10} = (0.1010\cdots)_2$ 。

(3) 含整数、小数部分的数的转换

只要将整数部分和小数部分分别进行转换, 得到转换后相应的整数和小数部分, 然后再将这两部分组合起来得到一个完整的数。

例 2.7 将十进制数 135.6875 分别转换成二进制数和八进制数。

解 只要将例 2.4 和例 2.5 的结果合起来就可, 即

$$(135.6875)_{10} = (10000111.1011)_2 = (207.54)_8$$

3. 二、八、十六进制数的相互转换

(1) 八进制数转换成二进制数

八进制数转换成二进制数的方法很简单, 只要把每一个八进制数字改写成等值的 3 位二进制数即可, 且保持高低位的次序不变。八进制数字与二进制数的对应关系如下。

$$(0)_8 = 000 \quad (1)_8 = 001 \quad (2)_8 = 010 \quad (3)_8 = 011$$

$$(4)_8 = 100 \quad (5)_8 = 101 \quad (6)_8 = 110 \quad (7)_8 = 111$$

例 2.8 将 $(13.724)_8$ 转换成二进制数。

解 $(13.724)_8 = (001\ 011.111\ 010\ 100)_2 = (1011.1110101)_2$

(2) 十六进制数转换成二进制数

十六进制数转换成二进制数的方法与八进制数转换成二进制数的方法类似, 只要把每一个十六进制数字改写成等值的 4 位二进制数即可, 且保持高低位的次序不变。十六进制数字与二进制数的对应关系如下。

$$(0)_{16} = 0000 \quad (1)_{16} = 0001 \quad (2)_{16} = 0010 \quad (3)_{16} = 0011$$

$$(4)_{16} = 0100 \quad (5)_{16} = 0101 \quad (6)_{16} = 0110 \quad (7)_{16} = 0111$$

$$(8)_{16} = 1000 \quad (9)_{16} = 1001 \quad (A)_{16} = 1010 \quad (B)_{16} = 1011$$

$$(C)_{16} = 1100 \quad (D)_{16} = 1101 \quad (E)_{16} = 1110 \quad (F)_{16} = 1111$$

例 2.9 将十六进制数 $(2B.5E)_{16}$ 转换成二进制数。

解 $(2B.5E)_{16} = (0010\ 1011.0101\ 1110)_2 = (101011.01011110)_2$

(3) 二进制数转换成八进制数

二进制数转换成八进制数时, 整数部分从低位向高位方向每 3 位用一个等值的八进制数来替换, 最后不足 3 位时在高位补 0 凑满 3 位; 小数部分从高位向低位方向每 3 位用一个等值的八进制数来替换, 最后不足 3 位时在低位补 0 凑满 3 位。例如:

$$(0.10101)_2 = (000.101\ 010)_2 = (0.52)_8$$

$$(10011.01)_2 = (010\ 011.010)_2 = (23.2)_8$$

(4) 二进制数转换成十六进制数

二进制数转换成十六进制数时, 整数部分从低位向高位方向每 4 位用一个等值的十六进制数来替换, 最后不足 4 位时在高位补 0 凑满 4 位; 小数部分从高位向低位方向每 4 位用一个等值的十六进制数来替换, 最后不足 4 位时在低位补 0 凑满 4 位。例如:

$$(11001.11)_2 = (0001\ 1001.1100)_2 = (19.C)_{16}$$

从以上可以看出,二进制数与八进制数、二进制数与十六进制数之间有很简单直观的对应关系。二进制数太长,书写、阅读均不方便;八进制数和十六进制数却像十进制数一样简练,易写易记。虽然计算机中只使用二进制一种计数制,但为了在开发和调试程序、查看机器代码时便于书写和阅读,人们经常使用八进制或十六进制来等价地表示二进制,所以大家也必须熟练掌握八进制和十六进制数的表示及其与二进制数之间的转换。

2.1.3 定点与浮点表示

日常生活中所使用的数有整数和实数之分,整数的小数点固定在数的最右边,可以省略不写,而实数的小数点则不固定。计算机中只能表示0和1,无法表示小数点,因此,要使得计算机能够处理日常使用的数值数据,必须要解决小数点的表示问题。通常计算机中通过约定小数点的位置来实现。小数点位置约定在固定位置的数称为定点数,小数点位置约定为可浮动的数称为浮点数。

1. 定点表示

定点表示法用来对定点小数和定点整数进行表示。对于定点小数,其小数点总是固定在数的左边,一般用来表示浮点数的尾数部分。对于定点整数,其小数点总是固定在数的最右边,因此可用“定点整数”来表示整数。

2. 浮点表示

对于任意一个实数 X ,可以表示成如下形式:

$$X = (-1)^S \times M \times R^E$$

其中 S 取值为0或1,用来决定数 X 的符号; M 是一个二进制定点小数,称为数 X 的尾数(mantissa); E 是一个二进制定点整数,称为数 X 的阶或指数(exponent); R 是基数(radix、base),可以取值为2、4、16等。在基数 R 一定的情况下,尾数 M 的位数反映数 X 的有效位数,它决定了数的表示精度,有效位数越多,表示精度就越高;阶 E 的位数决定数 X 的表示范围;阶 E 的值确定了小数点的位置。

从浮点数的形式来看,绝对值最小的非零数是如下形式的数: $0.0\cdots 01 \times R^{-(11\cdots 1)}$,而绝对值最大的数的形式应为: $0.11\cdots 1 \times R^{(11\cdots 1)}$,所以,假设 m 和 n 分别表示阶和尾数的位数,基数为2,则浮点数 X 的绝对值的范围为:

$$2^{-(2^m-1)} \times 2^{-n} \leq |X| \leq (1-2^{-n}) \times 2^{(2^m-1)}$$

上述公式中,紧靠 $|X|$ 左右两边的两个因子就是非零定点小数的绝对值表示范围,浮点数的最小数是定点小数的最小数 2^{-n} 去除以一个很大的数 $2^{(2^m-1)}$,而浮点数的最大数则是定点小数的最大数 $(1-2^{-n})$ 去乘以这个数 $2^{(2^m-1)}$,由此可见,浮点数的表示范围比定点数要大得多。

2.1.4 定点数的编码表示

定/浮点表示解决了小数点的表示问题。但是,对于一个数值数据来说,还有一个正/负号的表示问题。计算机中只能表示0和1,因此,正/负号也用0和1来表示。这种将数的符号用0和1表示的处理方式称为符号数字化。一般规定0表示正号,1表示负号。

数字化了的符号能否和数值部分一起参加运算呢?为了解决这个问题,就产生了把符号位

和数值部分一起进行编码的各种方法。因为任意一个浮点数都可以用一个定点小数和一个定点整数来表示,所以,只需要考虑定点数的编码表示。主要有4种定点数编码表示方法:原码、补码、反码和移码。

通常将数值数据在计算机内部编码表示的数称为机器数,而机器数真正的值(即现实世界中带有正负号的数)称为机器数的真值。例如, -10 (-1010B) 用8位补码表示为 11110110 ,说明机器数 11110110B (F6H 或 $0\text{x}\text{F6}$) 的真值是 -10 ,或者说, -10 的机器数是 11110110B (F6H 或 $0\text{x}\text{F6}$)。根据定义可知,机器数一定是一个0/1序列,通常缩写成十六进制形式。

假设机器数 X 的真值 X_T 的二进制形式(即式中 $X'_i = 0$ 或 1) 如下:

$$X_T = \pm X'_{n-2} \cdots X'_1 X'_0 \quad (\text{当 } X \text{ 为定点整数时})$$

$$X_T = \pm 0.X'_{n-2} \cdots X'_1 X'_0 \quad (\text{当 } X \text{ 为定点小数时})$$

对 X_T 用 n 位二进制数编码后,机器数 X 表示为:

$$X = X_{n-1} X_{n-2} \cdots X_1 X_0$$

机器数 X 有 n 位,式中 $X_i = 0$ 或 1 ,其中,第一位 X_{n-1} 是数的符号,后 $n-1$ 位 $X_{n-2} \cdots X_1 X_0$ 是数值部分。数值数据在计算机内部的编码问题,实际上就是机器数 X 的各位 X_i 的取值与真值 X_T 的关系问题。

在上述对机器数 X 和真值 X_T 的假设条件下,下面介绍各种定点数的编码表示。

1. 原码表示法

一个数的原码表示由符号位直接跟数值位构成,因此,也称“符号-数值”(sign and magnitude)表示法。原码表示法中,正数和负数的编码表示仅符号位不同,数值部分完全相同。

原码编码规则如下:

① 当 X_T 为正数时, $X_{n-1} = 0$, $X_i = X'_i$ ($0 \leq i \leq n-2$)

② 当 X_T 为负数时, $X_{n-1} = 1$, $X_i = X'_i$ ($0 \leq i \leq n-2$)

原码0有两种表示形式:

$$[+0]_{\text{原}} = 0\ 00 \cdots 0$$

$$[-0]_{\text{原}} = 1\ 00 \cdots 0$$

根据原码定义可知,对于真值 -10 (-1010B),若用8位原码表示,则其机器数为 10001010B (8AH 或 $0\text{x}8\text{A}$);对于真值 -0.625 (-0.101B),若用8位原码表示,则其机器数为 11010000B (D0H 或 $0\text{x}\text{D0}$)。

可以看出,原码表示的优点是:与真值的对应关系直观、方便,因此与真值之间的转换简单,并且用原码实现乘除运算也比较简便。其缺点是:0的表示不唯一,给使用带来不便。更重要的是,原码加减运算规则复杂。在进行原码加减运算过程中,要判定是否是两个异号数相加或两个同号数相减,若是,则必须判定两个数的绝对值大小,根据判断结果决定结果符号,并用绝对值大的数减去绝对值小的数。现代计算机中不用原码来表示整数,只用定点原码小数来表示浮点数的尾数部分。

2. 补码表示法

补码表示可以实现加减运算的统一,即用加法来实现减法运算。在计算机中,补码用来表

示带符号整数。补码表示法也称“2-补码”(two's complement)表示法,由符号位后跟上真值的模 2^n 补码构成,因此,在介绍补码概念之前,先讲一下有关模运算的概念。

(1) 模运算

在模运算系统中,若 A 、 B 、 M 满足下列关系: $A = B + K \times M$ (K 为整数),则记为 $A \equiv B(\text{mod } M)$ 。即 A 、 B 各除以 M 后的余数相同,故称 B 和 A 为模 M 同余。也就是说在一个模运算系统中,一个数与它除以“模”后得到的余数是等价的。

钟表是一个典型的模运算系统,其模数为12。假定现在钟表时针指向10点,要将它拨向6点,则有以下两种拨法。

① 逆时针拨4格: $10 - 4 = 6$

② 顺时针拨8格: $10 + 8 = 18 \equiv 6(\text{mod } 12)$

所以在模12系统中, $10 - 4 \equiv 10 + (12 - 4) \equiv 10 + 8(\text{mod } 12)$ 。即 $-4 \equiv 8(\text{mod } 12)$ 。

我们称8是-4对模12的补码。同样有 $-3 \equiv 9(\text{mod } 12)$; $-5 \equiv 7(\text{mod } 12)$ 等。

由上述例子与同余的概念,可得出如下结论:对于某一确定的模,某数 A 减去小于模的另一数 B ,可以用 A 加上 $-B$ 的补码来代替。这就是为什么补码可以借助加法运算来实现减法运算的道理。

例2.10 假定在钟表上只能顺时针方向拨动时针,如何用顺拨的方式实现将10点倒拨4格,拨动后钟表上是几点?

解 钟表是一个模运算系统,其模为12。根据上述结论,可得

$$10 - 4 \equiv 10 + (12 - 4) \equiv 10 + 8 \equiv 6(\text{mod } 12)$$

因此,可从10点顺时针拨8(-4的补码)格来实现倒拨4格,最后是6点。 ■

例2.11 假定算盘只有4档,且只能做加法,则如何用该算盘计算 $9828 - 1928$ 的结果?

解 这个算盘是一个“4位十进制数”模运算系统,其模为 10^4 。根据上述结论,可得

$$9828 - 1928 \equiv 9828 + (10^4 - 1928) \equiv 9828 + 8072 \equiv 7900(\text{mod } 10^4)$$

因此,可用9828加8072(-1928的补码)来实现9828减1928的功能。 ■

显然,在只有4档的算盘上运算时,如果运算结果超过4位,则高位无法在算盘上表示,只能用低4位表示结果,留在算盘上的值相当于是除以 10^4 后的余数。推广到计算机内部, n 位运算部件就相当于只有 n 档的二进制算盘,其模就是 2^n 。

计算机中的存储、运算和传送部件都只有有限位,相当于有限档数的算盘,因此计算机中所表示的机器数的位数也只有有限位。两个 n 位二进制数在进行运算过程中,可能会产生一个多于 n 位的数。此时,计算机和算盘一样,也只能舍弃高位而保留低 n 位,这样做可能会产生两种结果。

① 剩下的低 n 位数不能正确表示运算结果,也即丢掉的高位是运算结果的一部分。例如,在两个同号数相加时,当相加得到的和超出了 n 位数可表示的范围时出现这种情况,我们称此时发生了溢出(overflow)现象。

② 剩下的低 n 位数能正确表示运算结果,也即高位的舍去并不影响其运算结果。在两个同号数相减或两个异号数相加时,运算结果就是这种情况。舍去高位的操作相当于“将一个多

于 n 位的数去除以 2^n ，保留其余数作为结果”的操作，也就是“模运算”操作。

(2) 补码的定义

根据上述同余概念和数的互补关系，可引出补码的表示：正数的补码是它本身；负数的补码等于模与该负数绝对值之差。因此，数 X_T 的补码可用如下公式表示：

① 当 X_T 为正数时， $[X_T]_{\text{补}} = X_T = M + X_T (\text{mod } M)$ 。

② 当 X_T 为负数时， $[X_T]_{\text{补}} = M - |X_T| = M + X_T (\text{mod } M)$ 。

综合①和②，得到以下结论：对于任意一个数 X_T ， $[X_T]_{\text{补}} = M + X_T (\text{mod } M)$ 。

对于具有一位符号位和 $n-1$ 位数值位的 n 位二进制整数的补码来说，其补码定义如下：

$$[X_T]_{\text{补}} = 2^n + X_T \quad (-2^{n-1} \leq X_T < 2^{n-1}, \text{mod } 2^n)$$

(3) 特殊数据的补码表示

通过以下例子来说明几个特殊数据的补码表示。

例 2.12 分别求出补码的位数为 n 和 $n+1$ 时 -2^{n-1} 的补码表示。

解 当补码的位数为 n 位时，其模为 2^n ，因此：

$$[-2^{n-1}]_{\text{补}} = 2^n - 2^{n-1} = 2^{n-1} (\text{mod } 2^n) = 10\cdots0 \quad (n-1 \text{ 个 } 0)$$

当补码的位数为 $n+1$ 位时，其模为 2^{n+1} ，因此：

$$[-2^{n-1}]_{\text{补}} = 2^{n+1} - 2^{n-1} = 2^n + 2^{n-1} (\text{mod } 2^{n+1}) = 110\cdots0 \quad (n-1 \text{ 个 } 0)$$

从该例可以看出，同一个真值在不同位数的补码表示中，其对应的机器数不同。因此，在给定编码表示时，一定要明确编码的位数。在机器内部，编码的位数就是机器中运算部件的位数。

例 2.13 设补码的位数为 n ，求 -1 的补码表示。

解 对于整数补码有：

$$[-1]_{\text{补}} = 2^n - 1 = 11\cdots1 \quad (n \text{ 个 } 1)$$

对于 n 位补码表示来说， 2^{n-1} 的补码为多少呢？根据补码定义，有：

$$[2^{n-1}]_{\text{补}} = 2^n + 2^{n-1} (\text{mod } 2^n) = 2^{n-1} = 10\cdots0 \quad (n-1 \text{ 个 } 0)$$

最高位为 1，说明对应的真值是负数，显然这与实际情况不符。由此可知，为什么在 n 位补码定义中，真值的取值范围包含 -2^{n-1} ，而不包含 2^{n-1} 。

例 2.14 求 0 的补码表示。

解 根据补码的定义，有：

$$[+0]_{\text{补}} = [-0]_{\text{补}} = 2^n \pm 0 = 100\cdots0 (\text{mod } 2^n) = 00\cdots0 \quad (n \text{ 个 } 0)$$

从上述结果可知，补码 0 的表示是唯一的。这带来了以下两个方面的好处：

① 减少了 $+0$ 和 -0 之间的转换。

② 少占用一个编码表示，使补码比原码能多表示一个最小负数。在 n 位原码定点数中， $100\cdots0$ 用来表示 -0 ，但在 n 位补码表示中， -0 和 $+0$ 都用 $00\cdots0$ 表示，所以可用 $100\cdots0$ 来表示最小负整数 -2^{n-1} 。

(4) 补码与真值之间的转换方法

原码与真值之间的对应关系简单，只要对符号转换，数值部分不需改变。但对于补码来说，正数和负数的转换方式不同。根据定义，求一个正数的补码时，只要将正号“+”转换

为0, 数值部分无需改变; 求一个负数的补码时, 需要做减法运算, 因而不方便和直观。

例 2.15 设补码的位数为8, 求1101100和-1101100的补码表示。

解 补码的位数为8, 说明补码数值部分有7位, 故:

$$\begin{aligned} [1101100]_{\text{补}} &= 2^8 + 1101100 = 100000000 + 1101100 \pmod{2^8} = 01101100 \\ [-1101100]_{\text{补}} &= 2^8 - 1101100 = 100000000 - 1101100 \\ &= 10000000 + 10000000 - 1101100 \\ &= 10000000 + (1111111 - 1101100) + 1 \\ &= 10000000 + 0010011 + 1 \pmod{2^8} = 10010100 \end{aligned}$$

本例中是两个绝对值相同、符号相反的数。其中, 负数的补码计算过程中第一个10000000用于产生最后的符号1, 而第二个10000000拆为1111111+1, 而(1111111-1101100)实际是将真值各位取反。模仿这个计算过程, 不难从补码的定义推导出负数补码计算的一般步骤为: 符号位为1, 对真值部分“各位取反, 末尾加1”。

因此, 可以用以下简单方法求一个数的补码: 对于正数, 符号位取0, 其余各位同真值中对应各位; 对于负数, 符号位取1, 其余各位由真值“各位取反, 末尾加1”得到。

例 2.16 假定补码位数为8, 用简便方法求 $X = -1100011$ 的补码表示。

解 $[X]_{\text{补}} = 1\ 0011100 + 0\ 0000001 = 1\ 0011101$

对于由负数补码求真值的简便方法, 可以通过以上由真值求负数补码的计算方法得到。可以直接想到的方法是, 对补码数值部分先减1然后再取反。也就是说, 通过计算1111111-(0011101-1)得到, 该计算可以变为(1111111-0011101)+1, 亦即进行“取反加1”操作。因此, 由补码求真值的简便方法为: 若符号位为0, 则真值的符号为正, 其数值部分不变; 若符号位为1, 则真值的符号为负, 其数值部分的各位由补码“各位取反, 末尾加1”所得。

例 2.17 已知 $[X_T]_{\text{补}} = 1\ 0110100$, 求真值 X_T 。

解 $X_T = -(1001011 + 1) = -1001100$

根据上述有关补码和真值转换规则, 不难发现, 根据补码 $[X_T]_{\text{补}}$ 求 $[-X_T]_{\text{补}}$ 的方法是: 对 $[X_T]_{\text{补}}$ “各位取反, 末尾加1”。这里要注意最小负数取负后会发生溢出。

例 2.18 已知 $[X_T]_{\text{补}} = 1\ 0110100$, 求 $[-X_T]_{\text{补}}$ 。

解 $[-X_T]_{\text{补}} = 0\ 1001011 + 0\ 0000001 = 0\ 1001100$

例 2.19 已知 $[X_T]_{\text{补}} = 1\ 0000000$, 求 $[-X_T]_{\text{补}}$ 。

解 $[-X_T]_{\text{补}} = 0\ 1111111 + 0\ 0000001 = 1\ 0000000$ (结果溢出)

例2.19中出现了“两个正数相加, 结果为负数”的情况, 因此, 结果是一个错误的值, 我们称结果“溢出”, 该例中, 8位整数补码10000000对应的是最小负数 -2^7 , 对其取负后的值为 2^7 (即128), 而8位整数补码能表示的最大正数为 $2^7 - 1 = 127$, 因而数128太大, 无法用8位补码表示, 结果溢出。在结果溢出时, 有的编译器不会做任何提示, 因而可能会得到意想不到的结果。

(5) 变形补码

为了便于判断运算结果是否溢出, 某些计算机中还采用了一种双符号位的补码表示方式,

称为变形补码,也称为模4补码。在双符号位中,左符是真正的符号位,右符用来判别溢出。

假定变形补码的位数为 $n+1$ (其中符号占2位,数值部分占 $n-1$ 位),则变形补码可如下表示:

$$[X_T]_{\text{变补}} = 2^{n+1} + X_T \quad (-2^{n-1} \leq X_T < 2^{n-1}, \text{mod } 2^{n+1})$$

例 2.20 已知 $X_T = -1011$, 分别求出变形补码取6位和8位时的 $[X_T]_{\text{变补}}$ 。

解 $[X_T]_{\text{变补}} = 2^6 - 1011 = 100\ 0000 - 00\ 1011 = 11\ 0101$

$[X_T]_{\text{变补}} = 2^8 - 1011 = 100\ 000000 - 00\ 001011 = 11\ 110101$ ■

3. 反码表示法

负数的补码可采用“各位求反,末尾加1”的方法得到,如果仅各位求反而末尾不加1,那么就可得到负数的反码表示,因此负数反码的定义就是在相应的补码表示中再末尾减1。

反码表示存在以下几个方面的不足:0的表示不唯一;表数范围比补码少一个最小负数;运算时必须考虑循环进位。因此,反码在计算机中很少被使用,有时用作数码变换的中间表示形式。

4. 移码表示法

浮点数实际上是用两个定点数来表示的。用一个定点小数表示浮点数的尾数,用一个定点整数表示浮点数的阶(即指数)。一般情况下,浮点数的阶都用一种称之为“移码”的编码方式表示。通常将阶的编码表示称为阶码。

为什么要用移码表示阶呢?因为阶可以是正数,也可以是负数,当进行浮点数的加减运算时,必须先“对阶”(即比较两个数的阶的大小并使之相等)。为简化比较操作,使操作过程不涉及阶的符号,可以对每个阶都加上一个正的常数,称为偏置常数(bias),使所有阶都转换为正整数,这样,在对浮点数的阶进行比较时,就是对两个正整数进行比较,因而可以直观地将两个数按位从左到右进行比对,简化了对阶操作。

假设用来表示阶 E 的移码的位数为 n ,则 $[E]_{\text{移}} = \text{偏置常数} + E$,通常,偏置常数取 2^{n-1} 或 $2^{n-1} - 1$ 。

2.2 整数的表示

整数的小数点隐含在数的最右边,故无需表示小数点,因而也被称为定点数。计算机中的整数多用二进制表示,分为无符号整数(unsigned integer)和带符号整数(signed integer)两种。

2.2.1 无符号整数和带符号整数的表示

当一个编码的所有二进位都用来表示数值而没有符号位时,该编码表示的就是无符号整数。此时,默认数的符号为正,所以无符号整数就是正整数或非负整数。

一般在全部是正数且不出现负值结果的场合下,使用无符号整数。例如,可用无符号整数进行地址运算,或用来表示指针。通常把无符号整数简单地说是成无符号数。

由于无符号整数省略了一位符号位,所以在位数相同的情况下,它能表示的最大数比带符

号整数所能表示的大,例如,8位无符号整数的形式为00000000~11111111,对应的数的取值范围为 $0 \sim (2^8 - 1)$,即最大数为255,而8位带符号整数的最大数是127。

带符号整数也被称为有符号整数,它必须用一个二进制表示符号,虽然前面介绍的各种二进制编码表示都可以用来表示带符号整数,但是补码表示有其突出的优点,现代计算机中带符号整数都用补码表示。 n 位带符号整数的表示范围为 $-2^{n-1} \sim (2^{n-1} - 1)$ 。例如,8位带符号整数的表示范围为 $-128 \sim +127$ 。

2.2.2 C语言中的整数及其相互转换

C语言中支持多种整数类型。无符号整数在C语言中对应 unsigned short、unsigned int (unsigned)、unsigned long 等类型,通常在常数的后面加一个“u”或“U”来表示,例如,12345U,0x2B3Cu等;带符号整数在C语言中对应 short、int、long 等类型。

C语言标准规定了每种数据类型的最小取值范围,例如,int类型至少应为16位,取值范围为 $-32768 \sim 32767$ 。通常,short类型总是16位;int类型在32位和64位机器中都为32位;long类型在32位机器中为32位,在64位机器中为64位;long long类型是在ISO C99中引入的,规定它必须是64位。

小贴士

C语言是由贝尔实验室的Dennis M. Ritchie最早设计并实现的。为了使UNIX操作系统得以推广,1977年Dennis M. Ritchie发表了不依赖于具体机器的C语言编译文本《可移植的C语言编译程序》。1978年Brian W. Kernighan和Dennis M. Ritchie合著出版了《The C Programming Language》,从而使C语言成为目前世界上流行最广泛的高级程序设计语言之一。

1988年,随着微型计算机的日益普及,出现了许多C语言版本。由于没有统一的标准,使得这些C语言之间出现了一些不一致的地方。为了改变这种情况,美国国家标准学会(ANSI)为C语言制定了一套ANSI标准,对最初贝尔实验室的C语言做了重大修改。Brian W. Kernighan和Dennis M. Ritchie编写的《The C Programming Language》第2版对ANSI C做了全面的描述,该书被公认为是关于C语言的最好的参考手册之一。

国际标准化组织(ISO)接管了对C语言标准化的工作,在1990年推出了几乎和ANSI C一样的版本,称为“ISO C90”。该组织1999年又对C语言做了一些更新,成为“ISO C99”,该版本引进了一些新的数据类型,对英语以外的字符串本文提供了支持。

C语言中允许无符号整数和带符号整数之间的转换,转换前、后的机器数不变,只是转换前、后对其的解释发生了变化。转换后数的真值是将原二进制机器数按转换后的数据类型重新解释得到。例如,对于以1开头的的一个机器数,如果转换前是带符号整数类型,则其值为负整数,若将其转换为无符号数类型,则它被解释为一个无符号数,因而其值变成了一个大于等于 2^{n-1} 的正整数。也就是说,转换前的一个负整数,很可能转换后变成了一个值很大的正整数。由于上述原因,程序在某些情况下会发生意想不到的结果。例如,考虑以下C代码:

```

1 int x = -1;
2 unsigned u = 2147483648;
3
4 printf("x = %u = %d\n", x, x);
5 printf("u = %u = %d\n", u, u);

```

上述 C 代码中, x 为带符号整数, u 为无符号整数, 初值为 2 147 483 648 (即 2^{31})。函数 printf 用来输出数值, 格式符 %u、%d 分别用来以无符号整数和带符号整数的形式输出十进制数的值。当在一个 32 位机器上运行上述代码时, 它的输出结果如下。

```

x = 4294967295 = -1
u = 2147483648 = -2147483648

```

x 的输出结果说明如下: 因为 -1 的补码表示为 $11\cdots 1$, 所以当作为 32 位无符号数来解释 (格式符为 %u) 时, 其值为 $2^{32} - 1 = 4\,294\,967\,296 - 1 = 4\,294\,967\,295$ 。

u 的输出结果说明如下: 2^{31} 的无符号数表示为 $100\cdots 0$, 当这个数被解释为 32 位带符号整数 (格式符为 %d) 时, 其值为最小负数 $-2^{32-1} = -2^{31} = -2\,147\,483\,648$ (参见前面例 2.12)。

在 C 语言中, 如果执行一个运算时同时有无符号数和带符号整数参加, 那么, C 编译器会隐含地将带符号整数强制类型转换为无符号数, 因而会带来一些意想不到的结果。

例 2.21 在有些 32 位系统上, C 表达式 “ $-2147483648 < 2147483647$ ” 的执行结果为 false, 与事实不符; 但如果定义一个变量 “ $\text{int } i = -2147483648;$ ”, 表达式 “ $i < 2147483647$ ” 的执行结果却为 true。试分析产生上述结果的原因。如果将表达式写成 “ $-2147483647 - 1 < 2147483647$ ”, 则结果会怎样呢?

解 题目中遇到的问题在 ISO C90 标准下就会出现。在该标准下, 编译器在处理常量时, 会按 int32_t(int 型、long 型)、uint32_t(unsigned int、unsigned long)、int64_t(long long)、uint64_t(unsigned long long) 的顺序确定数据类型。也即, 在 $0 \sim 2^{31} - 1$ 之间的常量为 32 位带符号整型, 在 $2^{31} \sim 2^{32} - 1$ 之间为 32 位无符号整型, 在 $2^{32} \sim 2^{63} - 1$ 之间的为 64 位带符号整型, 在 $2^{63} \sim 2^{64} - 1$ 之间的为 64 位无符号整型。

编译器对于比较运算 (如 “ $<$ ”) 采用以下规则进行: 若两个操作数中至少有一个为无符号整型, 则采用无符号数比较, 只有当两个操作数均为带符号整型时才采用带符号整数比较。

编译器对 C 表达式 “ $-2147483648 < 2147483647$ ” 编译时, 将 “ -2147483648 ” 的表示分为两个部分来处理。对于 ISO C90 标准, 首先将 $2\,147\,483\,648 = 2^{31}$ 看成无符号整型, 其机器数为 $0x80000000$, 然后, 对其取负 (按位取反, 末位加 1), 结果仍为 $0x80000000$, 还是将其看成一个无符号整型, 其值仍为 $2\,147\,483\,648$ 。因而在处理条件表达式 “ $-2147483648 < 2147483647$ ” 时, 实际上是将 $2\,147\,483\,648$ 与 $2\,147\,483\,647$ 进行比较, 显然结果为 false。在计算机内部处理时, 真正进行的是对机器数 $0x80000000$ 和 $0x7FFFFFFF$ 做减法, 然后按照无符号整型来比较其大小。

没有符号位, 所有的位都取

编译器在处理 “ $\text{int } i = -2147483648;$ ” 时进行了类型转换, 将 $2\,147\,483\,648$ 转换为带符号整数后赋给变量 i , 其机器数还是 $0x80000000$, 但是值为 $-2\,147\,483\,648$, 执行 “ $i < 2147483647$ ” 时, 按照带符号整型来比较, 结果是 true。在计算机内部, 实际上是对机器数 $0x80000000$ 和 $0x7FFFFFFF$ 按照带符号整型进行比较。

对于“ $-2147483647 - 1 < 2147483647$ ”，编译器首先将 $2\ 147\ 483\ 647 = 2^{31} - 1$ (机器数为 $0x7FFFFFFF$) 看成带符号整型，然后对其取负，得到 $-2\ 147\ 483\ 647$ (机器数为 $0x80000001$)，然后将其减 1，得到 $-2\ 147\ 483\ 648$ ，与 $2\ 147\ 483\ 647$ 比较，得到结果为 true。在计算机内部，实际上是对机器数 $0x80000000$ 和 $0x7FFFFFFF$ 按照带符号整型进行比较。

在 ISO C99 标准下，C 表达式“ $-2147483648 < 2147483647$ ”的执行结果为 true。因为在该标准下，编译器在处理常量时会按 `int32_t` (int 型、long 型)、`int64_t` (long long)、`uint64_t` (unsigned long long) 的顺序确定数据类型。也即，在 $0 \sim 2^{31} - 1$ 之间的常量为 32 位带符号整型，在 $2^{31} \sim 2^{63} - 1$ 之间的为 64 位带符号整型，在 $2^{63} \sim 2^{64} - 1$ 之间的为 64 位无符号整型。因此，处理“ -2147483648 ”时，虽然分两步得到的机器数与在 ISO C90 标准中一样，都是 $0x80000000$ ，但是，因为其对应二进制数的值为 2^{31} ，在 $2^{31} \sim 2^{63} - 1$ 之间，因此被看成是 64 位带符号数，与 $2\ 147\ 483\ 647$ 比较时，按照带符号数进行比较，结果正确。 ■

2.3 浮点数的表示

计算机内部进行数据存储、运算和传送的部件位数有限，因而用定点数表示数值数据时，其表示范围很小。对于 n 位带符号整数，其表示范围为 $-2^{n-1} \sim (2^{n-1} - 1)$ ，运算结果很容易溢出，此外，用定点数也无法表示大量带有小数点的实数。因此，计算机中专门用浮点数来表示实数。

2.3.1 浮点数的表示范围

前面 2.1.3 节中提到，任意一个浮点数可用两个定点数来表示，用一个定点小数表示浮点数的尾数，用一个定点整数表示浮点数的阶。在 2.1.4 节中还提到，通常将阶的编码称为阶码，为便于对阶，阶码通常采用移码形式。

因为表示浮点数的两个定点数的位数是有限的，因而，浮点数的表示范围是有限的。以下例子说明可表示的浮点数位于数轴上的位置。

例 2.22 将十进制数 65 798 转换为下述 32 位浮点数格式。

0	1	8	9	31
符号	阶码	尾数		

其中，第 0 位为符号 S ；第 1~8 位为 8 位移码表示的阶码 E (偏置常数为 128)；第 9~31 位为 24 位二进制原码小数表示的尾数。基数为 2，规格化尾数形式为 $\pm 0.1bb \cdots b$ ，其中第一位“1”不明显表示出来，这样可用 23 个数位表示 24 位尾数。

解 因为 $(65\ 798)_{10} = (1\ 0000\ 0001\ 0000\ 0110)_2 = (0.1000\ 0000\ 1000\ 0011\ 0)_2 \times 2^{17}$ ，所以符号 $S=0$ ，阶码 $E = (128 + 17)_{10} = (145)_{10} = (1001\ 0001)_2$ 。

故 65 798 用该浮点数形式表示如下。

0	100 1000 1	000 0000 1000 0011 0000 0000
---	------------	------------------------------

用十六进制表示为 48808300H。 ■

上述格式的规格化浮点数的表示范围如下。

正数最大值： $0.11\cdots1 \times 2^{11\cdots1} = (1 - 2^{-24}) \times 2^{127}$ 。

正数最小值： $0.10\cdots0 \times 2^{00\cdots0} = (1/2) \times 2^{-128} = 2^{-129}$ 。

因为原码是对称的，故该浮点格式的范围是关于原点对称的，如图 2.2 所示。

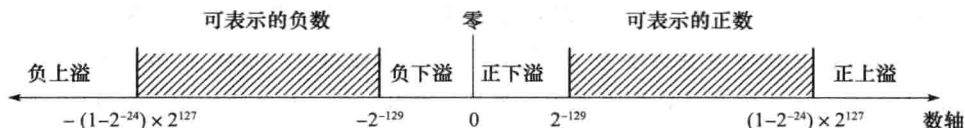


图 2.2 浮点数的表示范围

在图 2.2 中，数轴上有 4 个区间的数不能用浮点数表示。这些区间称为溢出区，接近 0 的区间为下溢区，向无穷大方向延伸的区间为上溢区。

根据浮点数的表示格式，只要尾数为 0，阶码取任何值其值都为 0，这样的数被称为机器零，因此机器零不唯一。通常用阶码和尾数同时为 0 来唯一表示机器零。即当结果出现尾数为 0 时，不管阶码为何值，都将阶码取为 0。也有的计算机将下溢区（阶码过小）的数近似成机器零。机器零有 +0 和 -0 之分。

2.3.2 浮点数的规格化

浮点数尾数的位数决定浮点数的有效数位，有效数位越多，数据的精度越高。为了在浮点数运算过程中尽可能多地保留有效数字的位数，使有效数字尽量占满尾数数位，必须在运算过程中对浮点数进行“规格化”操作。对浮点数的尾数进行规格化，除了能得到尽量多的有效数位以外，还可以使浮点数的表示具有唯一性。

从理论上来讲，规格化数的标志是真值的尾数部分中最高位具有非零数字。规格化操作有两种：左规和右规。当有效数位进到小数点前面时，需要进行右规，右规时，尾数每右移一位，阶码加 1，直到尾数变成规格化形式为止，右规时阶码会增加，因此阶码有可能溢出；当尾数出现形如 $\pm 0.0\cdots 0bb\cdots b$ 的运算结果时，需要进行左规，左规时，尾数每左移一位，阶码减 1，直到尾数变成规格化形式为止。

2.3.3 IEEE 754 浮点数标准

直到 20 世纪 80 年代初，浮点数表示格式还没有统一标准，不同厂商的计算机内部，其浮点数表示格式不同，在不同结构的计算机之间进行数据传送或程序移植时，必须进行数据格式的转换，而且，数据格式转换还会带来运算结果的不一致。因而，20 世纪 70 年代后期，IEEE 成立委员会着手制定浮点数标准，1985 年完成了浮点数标准 IEEE 754 的制定。其主要起草者是加州大学伯克利分校数学系教授 William Kahan，他帮助 Intel 公司设计了 8087 浮点处理器（FPU），并以此为基础形成了 IEEE 754 标准，Kahan 教授也因此获得了 1987 年的图灵奖。

目前几乎所有计算机都采用 IEEE 754 标准表示浮点数。在这个标准中，提供了两种基本浮点格式：32 位单精度格式和 64 位双精度格式，如图 2.3 所示。

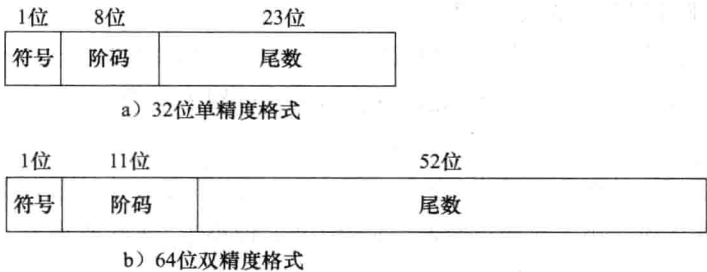


图 2.3 IEEE 754 浮点数格式

32 位单精度格式中包含 1 位符号 s 、8 位阶码 e 和 23 位尾数 f ；64 位双精度格式包含 1 位符号 s 、11 位阶码 e 和 52 位尾数 f 。其基数隐含为 2；尾数用原码表示，规格化尾数第一位总为 1，因而可在尾数中缺省第一位的 1，该缺省位称为隐藏位，隐藏一位后使得单精度格式的 23 位尾数实际上表示了 24 位有效数字，双精度格式的 52 位尾数实际上表示了 53 位有效数字。IEEE 754 规定隐藏位“1”的位置在小数点之前。

IEEE 754 标准中，阶码用移码形式，偏置常数并不是通常 n 位移码所用的 2^{n-1} ，而是 $(2^{n-1} - 1)$ ，因此，单精度和双精度浮点数的偏置常数分别为 127 和 1023。IEEE 754 的这种“尾数带一个隐藏位，偏置常数用 $(2^{n-1} - 1)$ ”的做法，不仅没有改变传统做法的计算结果，而且带来以下两个好处：

- ① 尾数可表示的位数多一位，因而使浮点数的精度更高。
- ② 阶码的可表示范围更大，因而使浮点数范围更大。

对于 IEEE 754 标准格式的数，一些特殊的位序列（如阶码为全 0 或全 1）有其特别的解释。表 2.2 给出了对各种形式的数的解释。

表 2.2 IEEE 754 浮点数的解释

值的类型	单精度 (32 位)				双精度 (64 位)			
	符号	阶码	尾数	值	符号	阶码	尾数	值
正零	0	0	0	0	0	0	0	0
负零	1	0	0	-0	1	0	0	-0
正无穷大	0	255(全 1)	0	∞	0	2047(全 1)	0	∞
负无穷大	1	255(全 1)	0	$-\infty$	1	2047(全 1)	0	$-\infty$
无定义数 (非数)	0 或 1	255(全 1)	$\neq 0$	NaN	0 或 1	2047(全 1)	$\neq 0$	NaN
规格化非零正数	0	$0 < e < 255$	f	$2^{e-127} (1.f)$	0	$0 < e < 2047$	f	$2^{e-1023} (1.f)$
规格化非零负数	1	$0 < e < 255$	f	$-2^{e-127} (1.f)$	1	$0 < e < 2047$	f	$-2^{e-1023} (1.f)$
非规格化正数	0	0	$f \neq 0$	$2^{-126} (0.f)$	0	0	$f \neq 0$	$2^{-1022} (0.f)$
非规格化负数	1	0	$f \neq 0$	$-2^{-126} (0.f)$	1	0	$f \neq 0$	$-2^{-1022} (0.f)$

在表 2.2 中，对 IEEE 754 中规定的数进行了以下分类。

1. 全 0 阶码全 0 尾数：+0/-0

IEEE 754 的零有两种表示：+0 和 -0。零的符号取决于符号 s 。一般情况下 +0 和 -0 是等效的。

2. 全 0 阶码非 0 尾数：非规格化数

非规格化数的特点是阶码为全 0，尾数高位有一个或几个连续的 0，但不全为 0。因此非规格化数的隐藏位为 0，并且单精度和双精度浮点数的阶分别为 -126 或 -1022 ，故浮点数的值分别为 $(-1)^s \times 0.f \times 2^{-126}$ 和 $(-1)^s \times 0.f \times 2^{-1022}$ 。

非规格化数可用于处理阶码下溢，使得出现比最小规格化数还小的数时程序也能继续下去。当运算结果的阶太小（比最小能表示的阶还小，即小于 -126 或小于 -1022 ）时，尾数右移 1 次，阶码加 1，如此循环直到尾数为 0 或阶达到可表示的最小值（ -126 或 -1022 ）。这个过程称为“逐级下溢”。因此，“逐级下溢”的结果就是使尾数变为非规格化形式，阶变为最小负数。例如，当一个十进制运算系统的最小阶为 -99 时，以下情况需进行阶的逐级下溢。

$$2.0000 \times 10^{-26} \times 5.2000 \times 10^{-84} = 1.04 \times 10^{-109} \rightarrow 0.1040 \times 10^{-108} \rightarrow 0.0104 \times 10^{-107} \rightarrow \dots \rightarrow 0.0$$

$$2.0002 \times 10^{-98} - 2.0000 \times 10^{-98} = 2.0000 \times 10^{-102} \rightarrow 0.2000 \times 10^{-101} \rightarrow 0.0200 \times 10^{-100} \rightarrow 0.0020 \times 10^{-99}$$

图 2.4 表示了加入非规格化数前、后 IEEE 754 的表数范围的变化。图中将可表示数以 $[2^n, 2^{n+1}]$ 的区间分组。区间 $[2^n, 2^{n+1}]$ 内所有数的阶相同，都为 n ，而尾数部分的变化范围为 $1.00\dots 0 \sim 1.11\dots 1$ ，这里小数点前的 1 是隐藏的隐含位。对于 32 位单精度规格化数，因为尾数的位数有 23 位，故每个区间内的数的个数相同，都是 2^{23} 个。例如，在正数范围内最左边的区间为 $[2^{-126}, 2^{-125}]$ ，在该区间内，最小规格化数为 $1.00\dots 0 \times 2^{-126}$ ，最大规格化数为 $1.11\dots 1 \times 2^{-126}$ 。在该区间中的各个相邻数之间具有等距性，其距离为 $2^{-23} \times 2^{-126}$ ，该区间右边相邻的区间为 $[2^{-125}, 2^{-124}]$ ，区间内各相邻数间的距离为 $2^{-23} \times 2^{-125}$ 。由此可见，每个右边区间内相邻数间的距离总比左边一个区间的相邻数距离大一倍，因此越离原点近的区间内的数的间隙越小。图 2.4a 给出了未定义非规格化数时的情况，在图中可看出，在 0 和最小规格化数 2^{-126} 之间有一个间隙未被利用。图 2.4b 给出了定义非规格化数后的情况，非规格化数就是在 0 和 2^{-126} 之间增加的 2^{23} 个附加数，这些相邻附加数之间与区间 $[2^{-126}, 2^{-125}]$ 内的相邻数等距，所有非规格化数具有与区间 $[2^{-126}, 2^{-125}]$ 内的数相同的阶，即最小阶（ -126 ）。尾数部分的变化范围为 $0.00\dots 0 \sim 0.11\dots 1$ 。这里的隐含位为 0，这也是非规格化数的重要标志之一。

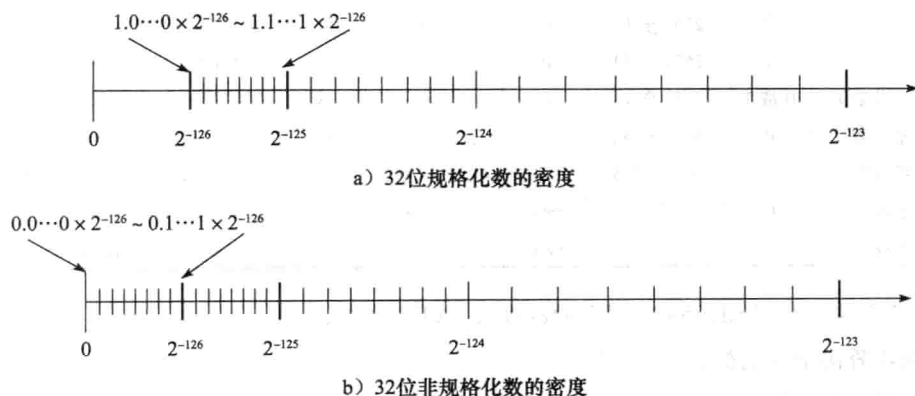


图 2.4 IEEE 754 中加入非规格化数前、后表数范围的变化

3. 全1阶码全0尾数: $+\infty / -\infty$

引入无穷大数使得在计算过程出现异常的情况下程序能继续进行下去, 并且可为程序提供错误检测功能。 $+\infty$ 在数值上大于所有有限数, $-\infty$ 则小于所有有限数, 无穷大数既可能是操作数, 也可能是运算的结果。当操作数为无穷大时, 系统可以有两种处理方式。

① 产生不发信号的非数 NaN。如 $+\infty + (-\infty)$ 、 $+\infty - (+\infty)$ 、 ∞ / ∞ 等。

② 产生明确的结果。如 $5 + (+\infty) = +\infty$ 、 $(+\infty) + (+\infty) = +\infty$ 、 $5 - (+\infty) = -\infty$ 、 $(-\infty) - (+\infty) = -\infty$ 等。

4. 全1阶码非0尾数: NaN

NaN (Not a Number) 表示一个没有定义的数, 称为非数。分为不发信号 (quiet) 和发信号 (signaling) 两种非数。有的书中把它们分别称为“静止的 NaN”和“通知的 NaN”。表 2.3 给出了能产生不发信号 (静止的) NaN 的计算操作。

引入 NaN 的目的是为了检测非初始化值的使用。程序员或编译程序可用非数表示每个浮点数变量的非初始化值。引入 NaN 还可以使计算出现异常时程序能继续进行下去, 让程序员将测试或判断延迟到方便的时候进行。可用尾数取值的不同来区分是“不发信号 NaN”还是“发信号 NaN”。当最高有效位为 1 时, 为不发信号 (静止的) NaN, 当结果产生这种非数时, 不发异常操作通知, 即不进行异常处理; 当最高有效位为 0

表 2.3 产生不发信号 NaN 的计算操作

运算类型	产生不发信号 NaN 的计算操作
所有	对通知 NaN 的任何计算操作
加减	无穷大相减: $(+\infty) + (-\infty)$ $(+\infty) - (+\infty)$ $(-\infty) + (+\infty)$ $(-\infty) - (-\infty)$
乘	$0 \times \infty$
除	$0/0$ 或 ∞/∞
求余	$x \bmod 0$ 或 $\infty \bmod y$
平方根	$\sqrt{x}$ 且 $x < 0$

时为发信号 (通知的) NaN, 当结果产生这种非数时, 则发一个异常操作通知, 表示要进行异常处理。因为 NaN 的尾数是非 0 数, 除了第一位有定义外其余的位没有定义, 所以可用其余位来指定具体的异常条件。一些没有数学解释的计算 (如 $0/0$, $0 \times \infty$ 等) 会产生一个非数 NaN。

5. 阶码非全0且非全1: 规格化非0数

对于阶码范围在 1~254 (单精度) 和 1~2046 (双精度) 的数, 是一个正常的规格化非 0 数。根据 IEEE 754 的定义, 这种数的阶的范围应该是 $-126 \sim +127$ (单精度) 和 $-1022 \sim +1023$ (双精度), 其值的计算公式分别为:

$$(-1)^s \times 1.f \times 2^{e-127} \text{ 和 } (-1)^s \times 1.f \times 2^{e-1023}$$

例 2.23 将十进制数 -0.75 转换为 IEEE 754 的单精度浮点数格式表示。

解 $(-0.75)_{10} = (-0.11)_2 = (-1.1)_2 \times 2^{-1} = (-1)^s \times 1.f \times 2^{e-127}$, 所以 $s=1$, $f=0.100\cdots 0$, $e=(127-1)_{10}=(126)_{10}=(0111\ 1110)_2$, 表示为单精度浮点数格式为 1 0111 1110 1000 0000... 0000 000, 用十六进制表示为 BF400000H。 ■

例 2.24 求 IEEE 754 单精度浮点数 C0A00000H 的值是多少。

解 求一个机器数的真值, 就是将该数转换为十进制数。首先将 C0A00000H 展开为一个 32 位单精度浮点数: 1 10000001 010 0000...0000。据 IEEE 754 单精度浮点数格式可知, 符号 $s=1$, $f=(0.01)_2=(0.25)_{10}$, 阶码 $e=(10000001)_2=(129)_{10}$, 所以, 其值为 $(-1)^s \times 1.f \times 2^{e-127} =$

$$(-1)^1 \times 1.25 \times 2^{129-127} = -1.25 \times 2^2 = -5.0。$$

IEEE 754 标准的单精度和双精度浮点数格式的特征参数见表 2.4。

表 2.4 IEEE 754 浮点数格式参数

参数	单精度浮点数	双精度浮点数	参数	单精度浮点数	双精度浮点数
字宽(位数)	32	64	尾数宽度	23	52
阶码宽度(位数)	8	11	阶码个数	254	2046
阶码偏置常数	127	1023	尾数个数	2^{23}	2^{52}
最大阶	127	1023	值的个数	1.98×2^{31}	1.99×2^{63}
最小阶	-126	-1022	数的量级范围	$10^{-38} \sim 10^{+38}$	$10^{-308} \sim 10^{+308}$

IEEE 754 用全 0 阶码和全 1 阶码表示一些特殊值, 如 0、 ∞ 和 NaN, 因此, 除去全 0 和全 1 阶码后, 单精度和双精度格式的阶码个数分别为 254 和 2046, 最大阶的值也相应地变为 127 和 1023。单精度规格化数的个数为 $254 \times 2^{23} = 1.98 \times 2^{31}$, 双精度规格化数的个数为 $2046 \times 2^{52} = 1.99 \times 2^{63}$ 。根据单精度和双精度格式的最大阶分别为 127 和 1023, 可以得出数的量级范围分别为 $10^{-38} \sim 10^{+38}$ 和 $10^{-308} \sim 10^{+308}$ 。

IEEE 754 除了对上述单精度和双精度浮点数格式进行了具体的规定以外, 还对单精度扩展和双精度扩展两种格式的最小长度和最小精度进行了规定。例如, IEEE 754 规定, 双精度扩展格式必须至少具有 64 位有效数字, 并总共占用至少 79 位, 但没有规定其具体的格式, 处理器厂商可以选择符合该规定的格式。

例如, SPARC 和 PowerPC 处理器中采用 128 位扩展双精度浮点数格式, 包含 1 位符号位 s 、15 位阶码 e (偏置常数为 16 383) 和 112 位尾数 f , 采用隐藏位, 所以有效位数为 113 位。

又如, Intel 及其兼容的 FPU 采用 80 位双精度扩展格式, 包含 4 个字段: 1 位符号位 s 、15 位阶码 e (偏置常数为 16 383)、1 位显式首位有效位 (explicit leading significant bit) j 和 63 位尾数 f 。Intel 采用的这种扩展浮点数格式与 IEEE 754 规定的单精度和双精度浮点数格式的一个重要的区别是, 它没有隐藏位, 有效位数共 64 位。

2.3.4 C 语言中的浮点数类型

C 语言中有 float 和 double 两种不同浮点数类型, 分别对应 IEEE 754 单精度浮点数格式和双精度浮点数格式, 相应的十进制有效数字分别为 7 位和 17 位左右。

C 对于扩展双精度的相应类型是 long double, 但是 long double 的长度和格式随编译器和处理器类型的不同而有所不同。例如, Microsoft Visual C++ 6.0 版本以下的编译器都不支持该类型, 因此, 用其编译出来的目标代码中 long double 和 double 一样, 都是 64 位双精度; 在 IA-32 上使用 gcc 编译器时, long double 类型数据采用 2.3.3 节中所述的 Intel x86 FPU 的 80 位双精度扩展格式表示; 在 SPARC 和 PowerPC 处理器上使用 gcc 编译器时, long double 类型数据采用 2.3.3 节中所述的 128 位双精度扩展格式表示。

当在 int、float 和 double 等类型数据之间进行强制类型转换时, 程序将得到以下数值转换结果 (假定 int 为 32 位)。

- ① 从 int 转换为 float 时, 不会发生溢出, 但有效数字可能被舍去。

② 从 int 或 float 转换为 double 时, 因为 double 的有效位数更多, 故能保留精确值。

③ 从 double 转换为 float 时, 因为 float 表示范围更小, 故可能发生溢出, 此外, 由于有效位数变少, 故数据可能被舍入。

④ 从 float 或 double 转换为 int 时, 因为 int 没有小数部分, 所以数据可能会向 0 方向被截断。例如, 1.9999 被转换为 1, -1.9999 被转换为 -1。此外, 因为 int 的表示范围更小, 故可能发生溢出。将大的浮点数转换为整数可能会导致程序错误, 这在历史上曾经有过惨痛的教训。

1996 年 6 月 4 日, Ariana 5 火箭初次航行, 在发射仅仅 37 秒钟后, 偏离了飞行路线, 然后解体爆炸, 火箭上载有价值 5 亿美元的通信卫星。根据调查发现, 原因是由于控制惯性导航系统的计算机向控制引擎喷嘴的计算机发送了一个无效数据。它没有发送飞行控制信息, 而是发送了一个异常诊断位模式数据, 表明在将一个 64 位浮点数转换为 16 位带符号整数时, 产生了溢出异常。溢出的值是火箭的水平速率, 这比原来的 Ariana 4 火箭所能达到的速率高出了 5 倍。在设计 Ariana 4 火箭软件时, 设计者确认水平速率决不会超出一个 16 位的整数, 但在设计 Ariana 5 时, 他们没有重新检查这部分, 而是直接使用了原来的设计。

在不同数据类型之间转换时, 往往隐藏着一些不容易被察觉的错误, 这种错误有时会带来重大损失, 因此, 编程时要非常小心。

例 2.25 假定变量 i 、 f 、 d 的类型分别是 int、float 和 double, 它们可以取除 $+\infty$ 、 $-\infty$ 和 NaN 以外的任意值。请判断下列每个 C 语言关系表达式在 32 位机器上运行时是否永真。

① $i == (\text{int})(\text{float})i$

② $f == (\text{float})(\text{int})f$

③ $i == (\text{int})(\text{double})i$

④ $f == (\text{float})(\text{double})f$

⑤ $d == (\text{float})d$

⑥ $f == -(-f)$

⑦ $(d+f)-d == f$

解 ① 不是, int 有效位数比 float 多, i 从 int 型转换为 float 型时有效位数可能丢失。

② 不是, float 有小数部分, f 从 float 型转换为 int 型时小数部分可能会丢失。

③ 是, double 比 int 有更大的精度和范围, i 从 int 型转换为 double 型时数值不变。

④ 是, double 比 float 有更大的精度和范围, f 从 float 型转换为 double 型时数值不变。

⑤ 不是, double 比 float 有更大的精度和范围, 当 d 从 double 型转换为 float 型时可能丢失有效数字或发生溢出。

⑥ 是, 浮点数取负就是简单地将数符取反。

⑦ 不是, 例如, 当 $d = 1.79 \times 10^{308}$ 、 $f = 1.0$ 时, 左边为 0 (因为 $d+f$ 时 f 需向 d 对阶, 对阶后 f 的尾数有效数位被舍去而变为 0, 故 $d+f$ 仍然等于 d , 再减去 d 后结果为 0), 而右边为 1。

2.4 十进制数的表示

人们日常使用和熟悉的是十进制，使用计算机来处理数据时，在计算机外部（如，键盘输入、屏幕显示或打印输出）看到的数据基本上是十进制形式，因此，有时需要计算机内部能够表示和处理十进制数据，以方便直接进行十进制数的输入输出或直接用十进制数进行计算。

在计算机内部，可以采用数字 0~9 对应的 ASCII 码字符来表示十进制数，也可以采用二进制编码的十进制数（简称 BCD）来表示十进制数。

2.4.1 用 ASCII 码字符表示

把十进制数看成字符串，直接用 ASCII 码表示，0~9 分别对应 30H~39H。这种方式下，一位十进制数对应 8 位二进制数，一个十进制数在计算机内部需占用多个连续字节，因此在存取一个十进制数时，必须说明其在内存的起始地址和字节个数。

用 ASCII 码字符串方式来表示十进制数，方便了十进制数的输入输出，但是，由于这种表示形式中含有非数值信息（高 4 位编码），所以对十进制数的运算很不方便。如果要对这种形式的十进制数进行计算，则必须先转换为二进制数或用 BCD 码表示十进制数。

2.4.2 用 BCD 码表示

这种十进制数采用二进制编码，通过专门的十进制数运算指令进行处理。计算机中可有专门的逻辑线路在 BCD 码运算时使每 4 位二进制数按十进制进行处理。

每位十进制数的取值可以是 0~9 这 10 个数之一，因此，每一个十进制数位必须至少有 4 位二进制位来表示。而 4 位二进制位可以组合成 16 种状态，去掉 10 种状态后还有 6 种冗余状态，所以从 16 种状态中选取 10 种状态来表示十进制数位 0~9 的方法很多，因而存在多种 BCD 码方案。

1. 有权 BCD 码

有权 BCD 码是指表示每个十进制数位的 4 个二进制数位（称为基 2 码）都有一个确定的权。最常用的一种编码就是 8421 码，它选取 4 位二进制数按计数顺序的前 10 个代码与十进制数字相对应，每位的权从左到右分别为 8、4、2、1，因此称为 8421 码，也称自然 BCD 码，记为 NBCD 码。

2. 无权 BCD 码

无权 BCD 码是指表示每个十进制数位的 4 个基 2 码没有确定的权。在无权码方案中，用得较多的是余 3 码和格雷码。

一个十进制数通常用多个对应的 BCD 码组合表示，每个数字对应 4 位二进制，两个数字占一个字节，数符可用 1 位二进制表示（1 表示负数，0 表示正数），或用 4 位二进制表示，并放在数字串最后，通常用 1100 表示正号，用 1101 表示负号。例如，Pentium 处理器中的十进制数占 80 位，第一个字节中的最高位为符号位，后面的 9 个字节可表示 18 位十进制数。

2.5 非数值数据的编码表示

逻辑值、字符等数据都是非数值数据，在机器内部它们用一个二进制位串表示。

2.5.1 逻辑值

正常情况下，每个字或其他可寻址单位（字节，半字等）是作为一个整体数据单元看待的。但是，某些时候还需要将一个 n 位数据看成是由 n 个一位数据组成，每个取值为 0 或 1。例如，有时需要存储一个布尔或二进制数据阵列，阵列中的每项只能取值为 1 或 0；有时可能需要提取一个数据项中的某位进行诸如“置 1”或“清 0”等操作。当数据以这种方式看待时，就被认为是逻辑数据。因此 n 位二进制数可表示 n 个逻辑值。逻辑数据只能参加逻辑运算，并且是按位进行的，如按位“与”、按位“或”、逻辑左移、逻辑右移等。

逻辑数据和数值数据都是一串 0/1 序列，在形式上无任何差异，需要通过指令的操作码类型来识别它们。例如，逻辑运算指令处理的是逻辑数据，算术运算指令处理的是数值数据。

2.5.2 西文字符

西文由拉丁字母、数字、标点符号及一些特殊符号所组成，它们统称为“字符”（character）。所有字符的集合叫做“字符集”。字符不能直接在计算机内部进行处理，因而也必须对其进行数字化编码。字符集中每一个字符都有一个代码（即二进制编码的 0/1 序列），构成了该字符集的代码表，简称码表。码表中的代码具有唯一性。

字符主要用于外部设备和计算机之间交换信息。一旦确定了所使用的字符集和编码方法后，计算机内部所表示的二进制代码和外部设备输入、打印和显示的字符之间就有唯一的对应关系。

字符集有多种，每一个字符集的编码方法也多种多样。目前计算机中使用最广泛的西文字符集及其编码是ASCII 码，即美国标准信息交换码（American Standard Code for Information Interchange），ASCII 字符编码见表 2.5。

从表 2.5 中可看出，每个字符都由 7 个二进位 $b_6b_5b_4b_3b_2b_1b_0$ 表示，其中 $b_6b_5b_4$ 是高位部分， $b_3b_2b_1b_0$ 是低位部分。一个字符在计算机中实际上是用 8 位表示的。一般情况下，最高一位 b_7 为 0。在需要奇偶校验时，这一位可用于存放奇偶校验值，此时称这一位为奇偶校验位。从表 2.5 中可看出 ASCII 字符编码有以下两个规律。

① 字符 0~9 这 10 个数字字符的高三位编码为 011，低 4 位分别为 0000~1001。当去掉高三位时，低 4 位正好是 0~9 这 10 个数字的 8421 码。这样既满足了正常的排序关系，又有利于实现 ASCII 码与十进制数之间的转换。

② 英文字母字符的编码值也满足正常的字母排序关系，而且大、小写字母的编码之间有简单的对应关系，差别仅在 b_5 这一位上。若这一位为 0，则是大写字母；若为 1，则是小写字母。这使得大、小写字母之间的转换非常方便。

表 2.5 ASCII 码表

	$b_6b_5b_4 =$ 000	$b_6b_5b_4 =$ 001	$b_6b_5b_4 =$ 010	$b_6b_5b_4 =$ 011	$b_6b_5b_4 =$ 100	$b_6b_5b_4 =$ 101	$b_6b_5b_4 =$ 110	$b_6b_5b_4 =$ 111
$b_3b_2b_1b_0 = 0000$	NUL	DLE	SP	0	@	P	`	p
$b_3b_2b_1b_0 = 0001$	SOH	DC1	!	1	A	Q	a	q
$b_3b_2b_1b_0 = 0010$	STX	DC2	"	2	B	R	b	r
$b_3b_2b_1b_0 = 0011$	ETX	DC3	#	3	C	S	c	s
$b_3b_2b_1b_0 = 0100$	EOT	DC4	\$	4	D	T	d	t
$b_3b_2b_1b_0 = 0101$	ENQ	NAK	%	5	E	U	e	u
$b_3b_2b_1b_0 = 0110$	ACK	SYN	&	6	F	V	f	v
$b_3b_2b_1b_0 = 0111$	BEL	ETB	'	7	G	W	g	w
$b_3b_2b_1b_0 = 1000$	BS	CAN	(	8	H	X	h	x
$b_3b_2b_1b_0 = 1001$	HT	EM	)	9	I	Y	i	y
$b_3b_2b_1b_0 = 1010$	LF	SUB	*	:	J	Z	j	z
$b_3b_2b_1b_0 = 1011$	VT	ESC	+	;	K	[	k	
$b_3b_2b_1b_0 = 1100$	FF	FS	,	<	L	\	l	
$b_3b_2b_1b_0 = 1101$	CR	GS	-	=	M	]	m	
$b_3b_2b_1b_0 = 1110$	SO	RS	.	>	N	^	n	~
$b_3b_2b_1b_0 = 1111$	SI	US	/	?	O	_	o	DEL

2.5.3 汉字字符

中文信息的基本组成单位是汉字，汉字也是字符。但汉字是表意文字，一个字就是一个方块图形。计算机要对汉字信息进行处理，就必须对汉字本身进行编码，但汉字的总数超过 6 万字，数量巨大，给汉字在计算机内部的表示、汉字的传输与交换、汉字的输入和输出等带来了一系列问题。为了适应汉字系统各组成部分对汉字信息处理的不同需要，汉字系统必须处理以下几种汉字代码：输入码、内码、字模点阵码。

1. 汉字的输入码

键盘是面向西文设计的，一个或两个西文字符对应一个按键，因此使用键盘输入西文字符非常方便。汉字是大字符集，专门的汉字输入键盘由于键多、查找不便、成本高等原因而几乎无法采用。由于汉字字数多，无法使每个汉字与西文键盘上的一个键相对应，因此必须使每个汉字用一个或几个键来表示，这种对每个汉字用相应的按键进行的编码表示就称为汉字的“输入码”，又称外码。因此汉字的输入码的码元（即组成编码的基本元素）是西文键盘中的某个按键。

2. 字符集与汉字内码

汉字被输入到计算机内部后，就按照一种称为“内码”的编码形式在系统中进行存储、查找、传送等处理。对于西文字符，它的内码就是 ASCII 码。

为了适应计算机处理汉字信息的需要，1981 年我国颁布了《信息交换用汉字编码字符集·基本集》（GB 2312—80）。该标准选出 6763 个常用汉字，为每个汉字规定了标准代码，以供汉字信息在不同计算机系统之间交换使用。这个标准称为国标码，又称国标交换码。

GB 2312 国标字符集由三部分组成：第一部分是字母、数字和各种符号，包括英文、俄文、日文平假名与片假名、罗马字母、汉语拼音等共 687 个；第二部分为一级常用汉字，共 3755 个，按汉语拼音排列；第三部分为二级常用汉字，共 3008 个，因为不太常用，所以按偏旁部首排列。

GB 2312 国标字符集中为任意一个字符（汉字或其他字符）规定了一个唯一的二进制代码。码表由 94 行（十进制编号 0~93 行）、94 列（十进制编号 0~93 列）组成，行号称为区号，列号称为位号。每一个汉字或符号在码表中都有各自的位置，因此各有一个唯一的位置编码，该编码用字符所在的区号及位号的二进制代码表示，7 位区号在左，7 位位号在右，共 14 位，这 14 位代码就叫汉字的“区位码”。区位码指出了汉字在码表中的位置。

汉字的区位码并不是其国标码（即国标交换码）。由于信息传输的原因，每个汉字的区号和位号必须各自加上 32（即十六进制的 20H），这样区号和位号各自加上 32 后的相应的二进制代码才是它的“国标码”，因此在“国标码”中区号和位号还是各自占 7 位。在计算机内部，为了处理与存储的方便，汉字国标码的前后各 7 位分别用一个字节来表示，所以共需 2 个字节才能表示一个汉字。因为计算机中的中西文信息是混合在一起进行处理的，所以汉字信息如不予以特别的标识，它与单字节的 ASCII 码就会混淆不清，无法识别。这就是前面给出的第一个要考虑的因素。为了解决这个问题，采用的方法之一，就是使表示汉字的两个字节的最高位（ b_7 ）总等于 1。这种双字节（16 位）的汉字编码就是其中的一种汉字“机内码”（即汉字内码）。例如，汉字“大”的区号是 20，位号是 83，因此区位码为 14 53H(0001 0100 0101 0011B)，国标码为 34 73H(0011 0100 0111 0011B)，前面的 34H 和字符“4”的 ASCII 码相同，后面的 73H 和字符“s”的 ASCII 码相同，将每个字节的最高位各设为 1 后，就得到其机内码 B4 F3H(1011 0100 1111 0011B)。这样就不会和 ASCII 码混淆了。应当注意，汉字的区位码和国标码是唯一的、标准的，而汉字内码可能随系统的不同而有差别。

随着亚洲地区计算机应用的普及与深入，汉字字符集及其编码还在发展。国际标准 ISO/IEC 10646 提出了一种包括全世界现代书面语言文字所使用的所有字符的标准编码，每个字符用 4 个字节（称为 UCS-4）或 2 个字节（称为 UCS-2）来编码。我国（包括香港、台湾地区）与日本、韩国联合制订了一个统一的汉字字符集（CJK 编码），共收集了上述不同国家和地区的共约 2 万多汉字及符号，采用 2 字节（即 UCS-2）编码，现已被批准为国家标准（GB 13000）。美国微软公司在 Windows 操作系统（中文版）中也已采用了中西文统一编码，其中收集了中、日、韩三国常用的约 2 万汉字，称为“Unicode”（2 字节编码），它与 ISO/IEC 10646 的 UCS-2 编码一致。

汉字输入码与汉字内码、国标交换码完全是不同范畴的概念，不能把它们混淆起来。使用不同的输入编码方法输入同一个汉字时，在计算机内部得到的汉字内码是一样的。

3. 汉字的字模点阵码和轮廓描述

经过计算机处理后的汉字，如果需要在屏幕上显示出来或用打印机打印出来，则必须把汉字机内码转换成人们可以阅读的方块字形式。

每一个汉字的字形都必须预先存放在计算机内，一套汉字（例如 GB 2312 国标汉字字符集）的所有字符的形状描述信息集合在一起称为字形信息库，简称字库（font library）。不同的

字体（如宋体、仿宋、楷体、黑体等）对应着不同的字库。在输出每一个汉字时，计算机都要先到字库中去找到它的字形描述信息，然后把字形信息送到相应的设备输出。

汉字的字形主要有两种描述方法：字模点阵描述和轮廓描述。字模点阵描述是将字库中的各个汉字或其他字符的字形（即字模），用一个其元素由“0”和“1”组成的方阵（如 16×16 、 24×24 、 32×32 甚至更大）来表示，汉字或字符中有黑点的地方用“1”表示，空白处用“0”表示，我们把这种用来描述汉字字模的二进制点阵数据称为汉字的字模点阵码。汉字的轮廓描述方法比较复杂，它把汉字笔画的轮廓用一组直线和曲线来勾画，记下每一直线和曲线的数学描述公式，目前已有两类国际标准：Adobe Type1 和 True Type。这种用轮廓线描述字形的方式精度高，字形大小可以任意变化。

2.6 数据的宽度和存储

2.6.1 数据的宽度和单位

计算机内部任何信息都被表示成二进制编码形式。二进制数据的每一位（0 或 1）是组成二进制信息的最小单位，称为一个“比特”（bit），或称“位元”，简称“位”。比特是计算机中存储、运算和传输信息的最小单位。

每个西文字符需要用 8 个比特表示，而每个汉字需要用 16 个比特才能表示。在计算机内部，二进制信息的计量单位是“字节”（byte），也称“位组”。一个字节等于 8 个比特。

计算机中运算和处理二进制信息时使用的单位除了比特和字节之外，还经常使用“字”（word）作为单位。必须注意，不同的计算机，字的长度和组成不完全相同，有的由 2 个字节组成，有的由 4 个、8 个甚至 16 个字节组成。

在考察计算机性能时，一个很重要的指标就是机器的“字长”。平时所说的“某种机器是 16 位机或是 32 位机”，其中的 16、32 就是指字长。所谓“机器字长”通常是指 CPU 内部用于整数运算的数据通路的宽度。CPU 内部数据通路是指 CPU 内部的数据流经的路径以及路径上的部件，主要是 CPU 内部进行数据运算、存储和传送的部件，这些部件的宽度基本上要一致才能相互匹配。因此，“字长”等于 CPU 内部用于整数运算的运算器位数和通用寄存器宽度。

“字”和“字长”的概念不同，这一点请注意。“字”用来表示被处理信息的单位，用来度量各种数据类型的宽度。通常系统结构设计者必须考虑一台机器将提供哪些数据类型，每种数据类型提供哪几种宽度的数，这时就要给出一个基本的“字”的宽度。例如，Intel x86 微处理器中把一个字定义为 16 位。所提供的数据类型中，就有单字宽度的无符号整数和带符号整数（16 位）、双字宽度的无符号整数和带符号整数（32 位）等。而“字长”表示进行数据运算、存储和传送的部件的宽度，它反映了计算机处理信息的一种能力。“字”和“字长”的长度可以一样，也可以不一样。例如，在 Intel 微处理器中，从 80386 开始就至少都是 32 位机器了，即字长至少为 32 位，但其字的宽度都定义为 16 位，32 位称为双字。

表示二进制信息存储容量时所用的单位要比字节或字大得多，主要有以下几种单位词头。

K(Kilo): $1\text{KB} = 2^{10}\text{字节} = 1024\text{字节}$

M(Mega): $1\text{MB} = 2^{20}$ 字节 = 1 048 576 字节

G(Giga): $1\text{GB} = 2^{30}$ 字节 = 1 073 741 824 字节

T(Tera): $1\text{TB} = 2^{40}$ 字节 = 1 099 511 627 776 字节

P(Peta): $1\text{PB} = 2^{50}$ 字节 = 1 125 899 906 842 624 字节

E(Exa): $1\text{EB} = 2^{60}$ 字节 = 1 152 921 504 606 846 976 字节

Z(Zetta): $1\text{ZB} = 2^{70}$ 字节 = 1 180 591 620 717 411 303 424 字节

Y(Yotta): $1\text{YB} = 2^{80}$ 字节 = 1 208 925 819 614 629 174 706 176 字节

在描述距离、频率等数值时通常用 10 的幂次表示，因而在由时钟频率计算得到的总线带宽或外设数据传输率中，度量单位表示的也是 10 的幂次。为区分这种差别，通常用 K 表示 1024，用 k 表示 1000，而其他前缀字母均为大写，表示的大小由其上下文决定。

经常使用的带宽单位如下。

比特/秒 (b/s)，有时也写为 bps

千比特/秒 (kb/s): $1\text{kb/s} = 10^3\text{b/s} = 1000\text{bps}$

兆比特/秒 (Mb/s): $1\text{Mb/s} = 10^6\text{b/s} = 1000\text{kbps}$

吉比特/秒 (Gb/s): $1\text{Gb/s} = 10^9\text{b/s} = 1000\text{Mbps}$

太比特/秒 (Tb/s): $1\text{Tb/s} = 10^{12}\text{b/s} = 1000\text{Gbps}$

由于程序需要对不同类型、不同长度的数据进行处理，所以，计算机中底层机器级的数据表示必须能够提供相应的支持。比如，需要提供不同长度的整数和不同长度的浮点数表示，相应地需要有处理单字节、双字节、4 字节甚至是 8 字节整数的整数运算指令，以及能够处理 4 字节、8 字节浮点数的浮点数运算指令等。

C 语言支持多种格式的整数和浮点数表示。数据类型 char 表示单个字节，能用来表示单个字符，也可用来表示 8 位整数。类型 int 之前可加上 long 和 short，以提供不同长度的整数表示。表 2.6 给出了在典型的 32 位机器和 64 位机器上 C 语言中数值数据类型的宽度。大多数 32 位机器使用“典型”

表 2.6 C 语言中数值数据类型的宽度

C 声明	典型的 32 位机器	64 位机器
char	1	1
short int	2	2
int	4	4
long int	4	8
char*	4	8
float	4	4
double	8	8

方式。从表 2.6 可以看出，短整数为 2 个字节，普通 int 型整数为 4 个字节，而长整数的宽度与机器字长的宽度相同。指针（例如，一个声明为类型 char* 的变量）和长整数的宽度一样，也等于机器字长的宽度。一般机器都支持 float 和 double 两种类型的浮点数，分别对应 IEEE 754 单精度和双精度格式。

由此可见，对于同一类型的数据，并不是所有机器都采用相同的数据宽度，分配的字节数随处理器和编译器的不同而不同。

2.6.2 数据的存储和排列顺序

任何信息在计算机中用二进制编码后，得到的都是一串 0/1 序列，每 8 位构成一个字节，不同的数据类型具有不同的字节宽度。在计算机中存储数据时，数据从低位到高位排列可以

从左到右，也可以从右到左。所以，用“最左位”（leftmost）和“最右位”（rightmost）来表示数据中的数位时会发生歧义。因此，一般用最低有效位（Least Significant Bit，简称 LSB）和最高有效位（Most Significant Bit，简称 MSB）来分别表示数的最低位和最高位。对于带符号数，最高位是符号位，所以 MSB 就是符号位。这样，不管数是从左往右排，还是从右往左排，只要明确 MSB 和 LSB 的位置，就可以明确数的符号和数值。例如，数“5”在 32 位机器上用 int 类型表示时的 0/1 序列为“0000 0000 0000 0000 0000 0000 0000 0101”，其中最前面的一位 0 是符号位，即 MSB = 0，最后面的 1 是数的最低有效位，即 LSB = 1。

如果以字节为一个排列基本单位，那么 LSB 表示最低有效字节（Least Significant Byte），MSB 表示最高有效字节（Most Significant Byte）。现代计算机基本上都采用字节编址方式，即对存储空间的存储单元进行编号时，每个地址编号中存放一个字节。计算机中许多类型数据由多个字节组成，例如，int 和 float 型数据占用 4 个字节，double 型数据占用 8 个字节等，而程序中对每个数据只给定一个地址。例如，在一个按字节编址的计算机中，假定 int 型变量 *i* 的地址为 08 00H，*i* 的机器数为 01 23 45 67H，这 4 个字节 01H、23H、45H、67H 应该各有一个存储地址，那么，地址 08 00H 对应 4 个字节中的哪个字节的地址呢？这就是字节排列顺序问题。

在所有计算机中，多字节数据都被存放在连续的字节序列中。根据数据中各字节在连续字节序列中的排列顺序的不同，可有两种排列方式：大端（big endian）和小端（little endian），如图 2.5 所示。

		0800H	0801H	0802H	0803H	
大端方式		01H	23H	45H	67H	
		0800H	0801H	0802H	0803H	
小端方式		67H	45H	23H	01H	

图 2.5 大端方式和小端方式

大端方式将数据的最高有效字节存放在低地址单元中，将最低有效字节存放在高地址单元中，即数据的地址就是 MSB 所在的地址。IBM 360/370、Motorola 68k、MIPS、Sparc、HP PA 等机器都采用大端方式。

小端方式将数据的最高有效字节存放在高地址单元中，将最低有效字节存放在低地址单元中，即数据的地址就是 LSB 所在的地址。Intel 80x86、DEC VAX 等都采用小端方式。

有些微处理器芯片，如 Alpha 和 Motorola 的 PowerPC，能够运行在任意一种模式，只要在芯片加电启动时选择确定采用大端还是小端模式即可。每个计算机系统内部的数据排列顺序都是一致的，但在系统之间进行通信时可能会发生问题。在排列顺序不同的系统之间进行数据通信时，需要进行顺序转换。网络应用程序员必须遵守字节顺序的有关规则，以确保发送方机器将它的内部表示格式转换为网络标准，而接收方机器则将网络标准转换为自己的内部表示格式。

此外，像音频、视频和图像等文件格式或处理程序也都涉及字节顺序问题。如 GIF、PC Paintbrush、Microsoft RTF 等采用小端方式，Adobe Photoshop、JPEG、MacPaint 等采用大端

方式。

了解字节顺序的好处还在于调试底层机器级程序时，能够清楚每个数据的字节顺序，以便将一个机器数正确转换为真值。例如，以下是一个由反汇编器（反汇编是汇编的逆过程，即将指令的机器代码转换为汇编表示）生成的一行针对 IA-32 处理器的机器级代码表示文本。

```
80483d2: 89 85 a0 fe ff ff  mov %eax, 0xffffea0(%ebp)
```

该文本行中，“80483d2”代表存储地址，是十六进制表示形式，“89 85 a0 fe ff ff”是指令的机器代码，按顺序存放在地址 0x80483d2 开始的 6 个连续存储单元中，“mov %eax, 0xffffea0(%ebp)”是指令的汇编形式。对该指令所指出的第二操作数进行访问时，需要先计算出该操作数的有效地址，这个有效地址是通过将寄存器 %ebp 的内容与立即数“0xffffea0”（字节序列为 FFH、FFH、FEH 和 A0H）相加得到的。该指令中的立即数是一个补码表示的带符号整数，补码为“0xffffea0”的数的真值为 $-10110000\text{B} = -176$ ，也即第二操作数的有效地址是将寄存器 %ebp 的内容减 176 后得到的值。指令执行时，可直接取出指令机器代码的后 4 个字节作为计算有效地址所用的立即数，从指令代码中可看出，立即数在内存存放的字节序列为 A0H、FEH、FFH、FFH，正好与有效地址计算时实际所用的字节序列相反。显然，该处理器采用的是小端方式。在阅读这种小端方式计算机的机器代码时，要记住字节是按照相反的顺序显示的。

例 2.26 以下是一段 C 程序，其中函数 show_int 和 show_float 分别用于显示 int 型和 float 型数据的位序列，show_pointer 用于显示指针型数据的位序列。显示的结果都用十六进制形式表示，并按照从低地址到高地址的方向显示。

```
1 void test_show_bytes(int val)
2 {
3     int ival = val;
4     float fval = (float) ival;
5     int *pval = &ival;
6     show_int(ival);
7     show_float(fval);
8     show_pointer(pval);
9 }
```

上述程序在不同系统（Linux 和 NT 运行于 Intel Pentium II）上运行的结果见表 2.7。

表 2.7 程序在不同系统中的运行结果

系统	值	类型	字节（十六进制）
Linux	12345	int	39 30 00 00
NT	12345	int	39 30 00 00
Sun	12345	int	00 00 30 39
Alpha	12345	int	39 30 00 00
Linux	12345.0	float	00 E4 40 46
NT	12345.0	float	00 E4 40 46
Sun	12345.0	float	46 40 E4 00
Alpha	12345.0	float	00 E4 40 46
Linux	&ival	int *	3C FA FF BF
NT	&ival	int *	1C FF 44 02
Sun	&ival	int *	EF FF FC E4
Alpha	&ival	int *	80 FC FF 1F 01 00 00 00

请回答下列问题。

① 十进制数 12 345 用 32 位补码整数和 32 位浮点数表示的结果各是什么？

② 十进制数 12 345 的整数表示和浮点数表示中存在一段相同位序列，标记出这段位序列，并说明为什么会相同。对一个负数来说，其整数表示和浮点数表示中是否也一定会出现一段相同的位序列？为什么？

③ Intel Pentium II 采用的是小端方式还是大端方式？

④ Sun 和 Alpha 之间能否直接进行数据传送？为什么？

⑤ 在 Alpha 上，表中数据字节 30H 所存放的地址是什么？

解 ① 十进制数 12 345 用 32 位补码整数表示为 0000 0000 0000 0000 0011 0000 0011 1001，用 32 位浮点数表示为 0100 0110 0100 0000 1110 0100 0000 0000。用十六进制表示分别为 00003039H 和 4640E400H。

② 十进制数 12 345 的整数表示和浮点数表示中相同位序列为 1 0000 0011 1001。因为对正数来说，原码和补码的编码相同，所以其整数（补码表示）和浮点数尾数（原码表示）的有效数位一样。12 345 的有效数位是 11 0000 0011 1001。有效数位在定点整数中位于低位数值部分，在浮点数的尾数中位于高位部分。因为尾数中有一个隐含的 1，所以第一个有效数位 1 在浮点数中不表示出来，因此，相同的位序列就是后面的 13 位。

因为 IEEE 754 浮点数的尾数用原码表示，而整数用补码表示，负数的原码和补码表示不同，所以，对某一个负数来说，其整数表示和浮点数表示中不一定会有一段相同的位序列。

③ Linux 和 NT（运行在 Intel Pentium II）的存放方式与书写习惯顺序相反，故 Intel Pentium II 采用的是小端方式。

④ Sun 和 Alpha 之间不能直接进行数据传送。因为它们采用了不同的存放方式，Sun 是大端方式，这里 Alpha 设置的是小端方式。

⑤ 在 Alpha 上数据字节 30H 存放在地址 000000011FFFC81H 中。因为从 Alpha 输出的 int 型指针结果来看，Alpha 的存储地址占 64 位，30H 是 int 型数据 12345 的次低有效字节，小端方式下数据地址取 LSB 的地址，所以 30H 存放的地址应该是数据地址随后的那个地址。根据小端方式下存放结果和书写习惯顺序相反的规律，可知数据 12 345 的地址是 000000011FFFC80H，所以，随后的地址就是 000000011FFFC81H。 ■

2.7 数据的基本运算

在计算机内部由于运算部件的位数有限，很多情况下会出现意料之外的运算结果，有时两个正数相加会得到一个负数，有时关系表达式“ $x < y$ ”和“ $x - y < 0$ ”会产生不同的结果。例如，在 2.2.2 节中的例 2.21 中提到，有的编译器对于“ $-2147483648 < 2147483647$ ”的执行结果为 false，但是，如果定义一个变量“`int i = -2147483648;`”，那么，表达式“`i < 2147483647`”的执行结果就为 true。如果不了解计算机底层的运算机制，则很难明白为什么会出现这些问题。因此，作为一个程序员，即使不需要进行底层硬件层的设计工作，也应该明白有关数据表示及其运算等方面的基本原理。

计算机硬件的设计目标来源于软件需求,高级语言中用到的各种运算,通过编译成底层的算术运算指令和逻辑运算指令实现,这些底层运算指令能在机器硬件上直接被执行。

2.7.1 按位运算和逻辑运算

C语言中的按位运算有:符号“|”表示按位“OR”运算;符号“&”表示按位“AND”运算;符号“~”表示按位“NOT”运算;符号“^”表示按位“XOR”运算。按位运算的一个重要运用就是实现掩码(masking)操作,通过与给定的一个位模式进行按位与,可以提取所需要的位,然后可以对这些位进行“置1”、“清0”、“是否为1测试”或“是否为0测试”等,这里位模式被称为“掩码”。例如,表达式“0x0F&0x8C”的运算结果为“00001100”,即“0x0C”。这里通过掩码“0x0F”提取了一个字节“0x8C”的低4位。

C语言中的逻辑运算符有:符号“||”表示“OR”运算;符号“&&”表示“AND”运算;符号“!”表示“NOT”运算。

逻辑运算很容易和按位运算混淆,事实上它们的功能完全不同。逻辑运算是非数值计算,其操作数只有两个逻辑值:“True”和“False”,通常用非0数表示逻辑值True,而全0表示逻辑值False;而按位运算是一种数值运算,运算时将两个操作数中对应各二进位按照指定的逻辑运算规则逐位进行计算。例如,若 $x = \text{FAH}$, $y = \text{7BH}$,则 $x \wedge y = \text{81H}$, $\sim(x \wedge y) = \text{7EH}$,而 $!(x \wedge y) = \text{00H}$ 。也即等价于表达式“ $x == y$ ”的是“ $!(x \wedge y)$ ”,而不是“ $\sim(x \wedge y)$ ”。

2.7.2 左移运算和右移运算

C语言中提供了一组移位运算。移位操作有逻辑移位和算术移位两种。

逻辑移位不考虑符号位的问题,左移时,高位移出,低位补0;右移时,低位移出,高位补0。对于无符号整数的逻辑左移,如果最高位移出的是1,则发生溢出。

因为计算机内部的带符号整数都是用补码表示的,所以对于带符号整数的移位操作应采用补码算术移位方式。左移时,高位移出,低位补0,如果移出的高位不同于移位后的符号位,也即,若左移前、后符号位不同,则发生“溢出”;右移时,低位移出,高位补符号。

虽然C语言没有明确规定应该采用逻辑移位还是算术移位,但是,实际上许多机器和编译器都对无符号整数采用逻辑移位方式,而对带符号整数采用算术移位方式。因此,编译器只要根据移位操作数类型就可选择是逻辑还是算术移位。表达式“ $x << k$ ”表示对数 x 左移 k 位。事实上,对于左移来说,逻辑移位和算术移位的结果都一样,都是丢弃 k 个最高位,并在低位补 k 个0。表达式“ $x >> k$ ”表示对数 x 右移 k 位。

每左移一位,相当于数值扩大一倍,所以左移可能会发生溢出,左移 k 位,相当于数值乘以 2^k ;每右移一位,相当于数值缩小一半,右移 k 位,相当于数值除以 2^k 。

例 2.27 已知32位寄存器R1中存放的变量 x 的机器数为80 00 00 04H,请回答下列问题。

① 当 x 是unsigned int类型时, x 的值是多少? $x/2$ 的值是多少? $x/2$ 的机器数是什么? $2x$ 的值是多少? $2x$ 的机器数是什么?

② 当 x 是int类型时, x 的值是多少? $x/2$ 的值是多少? $x/2$ 的机器数是什么? $2x$ 的值是多少? $2x$ 的机器数是什么?

解 ① 当 x 是 unsigned int 类型时, x 是无符号数, 机器数 80 00 00 04H 的真值是:

$$+1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0100\text{B} = 2^{31} + 2^2$$

对于 $x/2$ 的情况:

一方面, 根据 x 的真值求得 $x/2$ 的值为 $(2^{31} + 2^2)/2 = 2^{30} + 2$; 另一方面, $x/2$ 的机器数可由 x 逻辑右移一位得到, 因此, $x/2$ 的机器数是:

$$0100\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010 = 40\ 00\ 00\ 02\text{H}$$

由上述机器数得 $x/2$ 的真值为 $2^{30} + 2$ 。因此, 根据 x 的真值求出的 $x/2$ 的值与对 x 的机器数逻辑右移得到的 $x/2$ 的值是一样的。

对于 $2x$ 的情况:

一方面, 根据 x 的真值求得 $2x$ 的值为 $(2^{31} + 2^2) \times 2 = 2^{32} + 2^3$; 另一方面, $2x$ 的机器数可由 x 逻辑左移一位得到, 因此, $2x$ 的机器数是:

$$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1000 = 00\ 00\ 00\ 08\text{H}$$

由上述机器数得 $2x$ 的真值为 $2^3 = 8$ 。显然, $2^{32} + 2^3$ 不等于 8, 说明结果溢出。

实际上, 对 x 左移时, 移出的一位为 1, 表明有效数据被丢弃, 结果将会溢出, 导致根据 x 的真值求出的 $2x$ 与由 x 逻辑左移得到的 $2x$ 的值不一样。其原因在于 $2x$ 的值 $(2^{32} + 2^3)$ 超出了最大可表示值 $(2^{32} - 1)$, 无法用 32 位表示。

② 当 x 是 int 类型时, x 是带符号数, 机器数 80000004H (用二进制补码表示) 为: 1000 0000 0000 0000 0000 0000 0000 0100。

根据由补码求真值的简便方法“若符号位为 1, 则真值的符号为负, 其数值部分的各位由补码中相应各位取反后末尾加 1 而得”, 得到 x 的真值为:

$$-0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1100\text{B} = -(2^{31} - 2^2)$$

对于 $x/2$ 的情况:

一方面, 根据 x 的真值可求得 $x/2$ 的值为 $-(2^{31} - 2^2)/2 = -(2^{30} - 2)$; 另一方面, $x/2$ 的机器数可通过对 x 算术右移一位得到, 因此, $x/2$ 的机器数是:

$$1100\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010 = \text{C0000002H}$$

由上述机器数得 $x/2$ 的真值为:

$$-0011\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110\text{B} = -(2^{30} - 2)$$

由此可见, 由 x 的真值求出的 $x/2$ 与由 x 的机器数算术右移一位得到的 $x/2$ 是一样的。

对于 $2x$ 的情况:

一方面, 根据 x 的真值求得 $2x$ 的值为 $-(2^{31} - 2^2) \times 2 = -(2^{32} - 2^3)$; 另一方面, $2x$ 的机器数可由 x 算术左移一位得到, 因此, $2x$ 的机器数是:

$$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1000 = 00000008\text{H}$$

由上述机器数得 $2x$ 的真值为 $2^3 = 8$ 。显然, $-(2^{32} - 2^3)$ 不等于 8, 说明结果溢出。

对 x 左移时, 移出的 1 不等于左移后的最高位 0, 表明有效数据被丢弃, 结果将会溢出, 导致根据 x 的真值求出的 $2x$ 与由 x 算术左移一位得到的 $2x$ 的值不一样。其原因在于 $2x$ 的真值 $-(2^{32} - 2^3)$ 比最小可表示数 -2^{31} 还要小, 无法用 32 位表示。 ■

2.7.3 位扩展运算和位截断运算

C语言中没有明确的位扩展运算符，但是在进行数据类型转换时，如果遇到一个短数向长数转换，就要进行位扩展运算了。进行位扩展时，扩展后的数值应保持不变。有两种位扩展方式：0扩展和符号扩展。0扩展用于无符号数，只要在短的无符号数前面添加足够的0即可。符号扩展用于补码表示的带符号整数，通过在短的带符号整数前添加足够多的符号位来扩展。

考虑以下C语言程序代码：

```
1 short si = -32768;
2 unsigned short usi = si;
3 int i = si;
4 unsigned ui = usi;
```

执行上述程序段，并在32位大端方式机器上输出变量*si*、*usi*、*i*、*ui*的十进制和十六进制值，可得到各变量的输出结果为：

```
si = -32768    80 00
usi = 32768    80 00
i = -32768    FF FF 80 00
ui = 32768    00 00 80 00
```

由此可见，-32768的补码表示和32768的无符号数表示具有相同的16位0/1序列，分别将它们扩展为32位后，得到的32位位序列的高位不同。因为前者是符号扩展，高16位补符号1，后者是0扩展，高16位补0。

位截断发生在将长数转换为短数时，例如，对于下列代码：

```
1 int i = 32768;
2 short si = (short)i;
3 int j = si;
```

在一台32位大端方式机器上执行上述代码段时，第2行要求强行将一个32位带符号整数截断为一个16位带符号整数，32768的32位补码表示为00008000H，截断为16位后变成8000H，它是-32768的16位补码表示。再将该16位带符号整数扩展为32位时，就变成了FFFF8000H，它是-32768的32位补码表示，因此*j*为-32768。也就是说，原来的*i*（值为32768）经过截断、再扩展后，其值变成了-32768，不等于原来的值了。

从上述例子可以看出，截断一个数可能会因为溢出而改变它的值。因为长数的表示范围远远大于短数的表示范围，所以当长数足够大到短数无法表示的程度，则截断时就会发生溢出。上述例子中的32768大于16位补码能表示的最大数32767，所以就发生了截断错误。这里所说的截断溢出和截断错误只会导致程序出现意外的计算结果，但并不导致任何异常或错误报告，因此，错误的隐蔽性很强，需要引起注意。

2.7.4 整数加减运算

在程序设计时通常把指针、地址等说明为无符号整数，因而在进行指针或地址运算时需要进行无符号整数的加减运算。而其他情况下，通常都是带符号整数运算。无符号整数和带符号

整数的加减运算电路是完全一样的，它们都可以在如图 2.6 所示的整数加减运算器中实现，图中加法器中进行的是无符号数加运算，MUX 是一个二路选择器，有关加法器和二路选择器的功能和结构请参看附录 A。

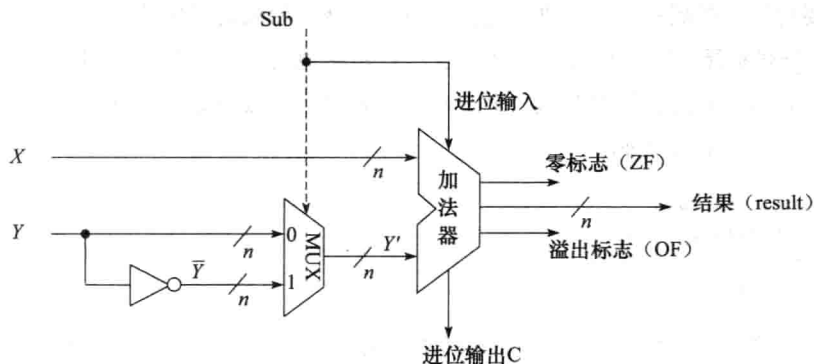


图 2.6 n 位整数加减运算器

图 2.6 中， X 和 Y 是两个 0/1 序列，对于带符号整数 x 和 y 来说， X 和 Y 就是 x 和 y 的补码表示，对于无符号整数 x 和 y 来说， X 和 Y 就是 x 和 y 的无符号数表示。不管是补码减法还是无符号数减法，都是用被减数加上减数的负数的补码来实现。根据求补公式，减数 y 的负数的补码 $[-y]_{\text{补}} = \bar{Y} + 1$ ，因此，只要在加法器的 Y' 输入端，加 n 个反向器以实现各位取反的功能，然后加一个 2 选 1 多路选择器 MUX，用一个控制端 Sub 来控制选择将原码 Y 输入到 Y' 端还是将 Y 各位取反后输入到 Y' 端，并将控制端 Sub 同时作为低位进位送到加法器。当 Sub 为 1 时做减法，即实现 $x - y = X + \bar{Y} + 1$ ；当 Sub 为 0 时做加法，即实现 $x + y = X + Y$ 。

图 2.6 给出了两个输出标志信息：零标志 ZF 和 溢出标志 OF。ZF = 1 表示结果为 0，因此当结果 (result) 的所有位都为 0 时，使 ZF = 1，否则 ZF = 0；OF = 1 表示带符号整数的加减运算发生溢出，因为两个同号数相加其结果的符号一定同两个加数的符号，所以，当 X 和 Y' 的最高位相同且不同于结果的最高位时，使 OF = 1，否则 OF = 0。

通常，在整数加减运算器的输出中，除了 ZF 和 OF 以外，还有两个常用标志位：符号标志 SF 和 进/借位标志 CF。其中，SF 表示带符号整数加减运算结果的符号位，因此，可以直接取 result 的最高位作为 SF。CF 用来表示无符号数加减运算时的进/借位。加法时，若 CF = 1 表示加法有进位；减法时，若 CF = 1 表示不够减。因此，加法时 CF 就应等于进位输出 C；减法时，应将进位输出 C 取反来作为借位标志。综合起来，可得： $CF = \text{Sub} \oplus C$ 。

例 2.28 以下是一个 C 语言程序，用来计算一个数组 a 中每个元素的和。当参数 len 为 0 时，返回值应该是 0，但是在机器上执行时，却发生了存储器访问异常。请问这是是什么原因造成的，并说明程序应该如何修改。

```
1 float sum_array(int a[], unsigned len)
2 {
3     int i, sum = 0;
4
5     for(i = 0; i <= len-1; i++)
6         sum += a[i];
```



```

7
8     return sum;
9 }

```

解 当 len 为 0 时, 在图 2.6 的电路中计算 $len - 1$, 此时 X 为 00000000H, Y 为 00000001H, $Sub = 1$, 因此计算出来的结果是 32 个 1。在对条件表达式 “ $i \leq len - 1$ ” 进行判断时, 通过做减法得到的标志信息来进行比较。开始时 $i = 0$, 因此, 在图 2.6 的电路中计算 $0 - \text{FFFFFFFFH}$, 此时, X 为 00000000H, Y 为 FFFFFFFFH, $Sub = 1$, 显然加法器的输出结果 $result$ 为 00000001H, 进位输出 $C = 0$, 因此借/进位标志 $CF == Sub \oplus C = 1$, 零标志 $ZF = 0$, 符号标志 $SF = 0$, 溢出标志 $OF = 0$ 。

因为中 len 是 `unsigned` 类型, 所以对条件表达式 “ $i \leq len - 1$ ” 进行判断时按照无符号数进行比较 (对应的是无符号整数比较并转移指令), 即根据 CF 的取值来判断大小, 当 $CF = 1$ 且 $ZF = 0$ 时表示小于, 显然, 任何无符号数都比 32 个 1 小, 因此循环体被不断执行, 最终导致数组访问越界而发生存储器访问异常。

正确的做法是将参数 len 声明为 `int` 型。这样, 虽然加法器中的运算以及生成的所有标志信息与 len 为无符号数时完全一样, 但是, 因为条件表达式 “ $i \leq len - 1$ ” 中的 i 和 len 都是带符号整数, 因而会按照带符号整数进行比较 (对应的是带符号整数比较并转移指令), 即根据 OF 和 SF 的值是否相同来判断大小, 当 $OF = SF$ 且 $ZF = 0$ 时表示大于。因此, 当 $i = 0$ 时, $len = 0$ 使得 “ $i \leq len - 1$ ” 条件不满足, 从而跳出循环执行。

无符号数加减运算在图 2.6 所示的电路中执行, 运算的结果取低 n 位, 相当于取模为 2^n , 也即当两数相加的结果大于 2^n , 则大于 2^n 的部分将被减掉。因此无符号数加法运算公式如下。

$$result = \begin{cases} x + y & (x + y < 2^n) \\ x + y - 2^n & (2^n \leq x + y < 2^{n+1}) \end{cases} \quad (2.1)$$

在图 2.6 所示电路上做无符号数减运算 $x - y$ 时, 用 x 加 $[-y]_{\text{补}}$ 来实现, 根据补码公式知, $[-y]_{\text{补}} = 2^n - y$, 因此, $result = x + (2^n - y) = x - y + 2^n$, 当 $x - y > 0$ 时, 2^n 被减掉。因此,无符号数减法运算公式如下。

$$result = \begin{cases} x - y & (x - y > 0) \\ x - y + 2^n & (x - y < 0) \end{cases} \quad (2.2)$$

例 2.29 假设 8 位无符号整数变量 x 和 y 的机器数分别是 X 和 Y , 相应加减运算在图 2.6 所示电路中执行。若 $X = A6H$, $Y = 3FH$, 则 x 、 y 、 $x + y$ 和 $x - y$ 的值分别是多少? 若 $X = A6H$, $Y = FFH$, 则 x 、 y 、 $x + y$ 和 $x - y$ 的值又分别是多少? (说明: 这里的 $x + y$ 和 $x - y$ 的值是指经过运算电路处理后得到的 $result$ 对应的值。)

解 若 $X = A6H$, $Y = 3FH$, 则 $x + y$ 的机器数 $X + Y = 10100110 + 00111111 = 11100101 = E5H$, $x - y$ 的机器数 $X - Y = 10100110 + 11000001 = 01100111 = 67H$ 。因此, x 、 y 、 $x + y$ 的 $result$ 和 $x - y$ 的 $result$ 分别是 166、63、229 和 103, 显然运算结果符合上述公式 (2.1) 和 (2.2)。

验证如下: 因为 $x + y = 166 + 63 < 2^8 = 256$, 因而 $x + y$ 的 $result$ 应该等于 $x + y = 166 + 63 = 229$; 因为 $x - y = 166 - 63 > 0$, 因而 $x - y$ 的 $result$ 应该等于 $x - y = 166 - 63 = 103$, 验证正确。

若 $X = A6H$, $Y = FFH$, 则 $X + Y = 10100110 + 11111111 = 10100101 = A5H$, $X - Y = 10100110 + 00000001 = 1010 0111 = A7H$ 。因此, x 、 y 、 $x + y$ 的 $result$ 和 $x - y$ 的 $result$ 分别是 166、255、165

和 167, 运算结果符合上述运算公式 (2.1) 和 (2.2)。

验证如下: 因为 $x+y=166+255>2^8=256$, 因而, $x+y$ 的 result 应该等于 $x+y-2^8=166+255-256=165$; 因为 $x-y=166-255<0$, 因而 $x-y$ 的 result 应该等于 $x-y+2^8=166-255+256=167$, 验证正确。 ■

带符号整数加法运算也在图 2.6 所示电路中执行。如果两个 n 位加数 x 和 y 的符号相反, 则一定不会溢出, 只有两个加数的符号相同时才可能发生溢出。两个加数都是正数时发生的溢出称为正溢出; 两个加数都是负数时发生的溢出称为负溢出。图 2.6 中实现的带符号整数加法运算公式如下:

$$\text{result} = \begin{cases} x+y-2^n & (2^{n-1} \leq x+y) & \text{正溢出} \\ x+y & (-2^{n-1} \leq x+y < 2^{n-1}) & \text{正常} \\ x+y+2^n & (x+y < -2^{n-1}) & \text{负溢出} \end{cases} \quad (2.3)$$

与无符号整数减法运算类似, 带符号整数减法也通过加法来实现, 同样也是用被减数加上减数的负数的补码来实现。图 2.6 中实现的带符号整数减法运算公式如下:

$$\text{result} = \begin{cases} x-y-2^n & (2^{n-1} \leq x-y) & \text{正溢出} \\ x-y & (-2^{n-1} \leq x-y < 2^{n-1}) & \text{正常} \\ x-y+2^n & (x-y < -2^{n-1}) & \text{负溢出} \end{cases} \quad (2.4)$$

例 2.30 假设 8 位带符号整数变量 x 和 y 的机器数分别是 X 和 Y , 相应加减运算在图 2.6 所示电路中执行。若 $X=A6H$, $Y=3FH$, 则 x 、 y 、 $x+y$ 和 $x-y$ 的值分别是多少? 若 $X=A6H$, $Y=FFH$, 则 x 、 y 、 $x+y$ 和 $x-y$ 的值又分别是多少? (说明: 这里的 $x+y$ 和 $x-y$ 的值是指经过运算电路处理后得到的 result 对应的值。)

解 若 $X=A6H$, $Y=3FH$, 则 $x+y$ 的机器数 $X+Y=10100110+00111111=11100101=E5H$, $x-y$ 的机器数 $X-Y=10100110+11000001=01100111=67H$ 。因为带符号整数用补码表示, 所以, x 、 y 、 $x+y$ 的 result 和 $x-y$ 的 result 分别是 -90、63、-27 和 103, 经验证, 运算结果符合上述公式 (2.3) 和 (2.4)。

验证如下: 因为 $-2^7 \leq x+y = -90+63 < 2^7$, 因而 $x+y$ 的 result 应该等于 $x+y = -90+63 = -27$; 因为 $x-y = -90-63 < -2^7$, 即负溢出, 因而 $x-y$ 的 result 应该等于 $x-y+2^8 = -90-63+256=103$, 验证正确。

若 $X=A6H$, $Y=FFH$, 则 $X+Y=10100110+11111111=10100101=A5H$, $X-Y=10100110+00000001=1010\ 0111=A7H$ 。 x 、 y 、 $x+y$ 的 result 和 $x-y$ 的 result 分别是 -90、-1、-91 和 -89, 经验证, 运算结果符合上述运算公式 (2.3) 和 (2.4)。

验证如下: 因为 $-2^7 \leq x+y = -90+(-1) < 2^7$, 因而, $x+y$ 的 result 应该等于 $x+y = -90+(-1) = -91$; 因为 $-2^7 \leq x-y = -90-(-1) < 2^7$, 因而 $x-y$ 的 result 应该等于 $x-y = -90-(-1) = -89$, 验证正确。 ■

例 2.29 和例 2.30 中给出的机器数 X 和 Y 完全相同, 在同样的电路中计算, 因而得到的和(差)的机器数也完全相同。对于同一个机器数, 作为无符号整数解释和作为带符号整数解释时的值不同, 因而例 2.29 和例 2.30 中得到的和(差)的值完全不同。从这里可以看出, 在电路中

执行运算时所有的数都只是一个0/1序列,在底层机器级这个层次上,并不区分操作数是什么类型。因而,在一些指令集体系结构中,加法指令和减法指令并不区分是无符号整数加(减)指令还是带符号整数的加(减)指令,例如Intel x86就是如此。但是,有些处理器指令系统,也会提供专门的带符号整数加(减)指令和专门的无符号整数加(减)指令,例如MIPS架构就是如此。

在Intel x86架构中,不管高级语言程序中定义的变量是带符号整数还是无符号整数类型,对应的加(减)指令都一样,都是在如图2.6所示的电路中执行。每条加(减)指令执行以后,总是把运算电路中结果的低 n 位(result)送到目的寄存器,同时根据运算结果产生相应的进/借位标志CF、符号标志SF、溢出标志OF和零标志ZF等,并保存到标志寄存器(FLAGS/EFLAGS)中。

在MIPS架构中,提供了专门的带符号整数的加(减)指令(如add、sub指令)和无符号整数的加(减)指令(如addu、subu指令),因而在MIPS处理器的运算电路中会区分是带符号整数还是无符号整数运算。不过,它们之间的不同也仅在于是否判断和处理溢出,带符号整数加减时会判断溢出并对溢出进行处理,而无符号整数加减时不判断溢出,对于其余部分的处理两者完全一样。

由于在底层的机器级层次对无符号整数和带符号整数的运算不加区分,因而,在高级语言程序执行过程中,带符号整数隐式地转换为无符号整数运算时,会出现像例2.21和例2.28中那样的意想不到的错误或存在漏洞。杜绝使用无符号整数可以避免这类问题。也有一些语言为避免这类问题,采用不支持无符号整数的方式,例如,Java语言就不支持无符号整数。

2.7.5 整数乘除运算

高级语言中两个 n 位整数相乘得到的结果通常也是一个 n 位整数,也即结果只取 $2n$ 位乘积中的低 n 位。例如,在C语言中,参加运算的两个操作数的类型和结果的类型必须一致,如果不一致则会先转换为一致的数据类型再进行计算。

根据二进制运算规则,在计算机算术中存在以下结论:假定两个 n 位无符号整数 x_u 和 y_u 对应的机器数为 X_u 和 Y_u , $p_u = x_u \times y_u$, p_u 为 n 位无符号整数且对应的机器数为 P_u ;两个 n 位带符号整数 x_s 和 y_s 对应的机器数为 X_s 和 Y_s , $p_s = x_s \times y_s$, p_s 为 n 位带符号整数且对应的机器数为 P_s 。若 $X_u = X_s$ 且 $Y_u = Y_s$,则 $P_u = P_s$ 。表2.8中给出了4位无符号整数和4位带符号整数乘法的部分例子,显然这些例子符合上述结论。

表2.8 4位无符号整数和4位带符号整数乘法示例

序号	运算	x	X	y	Y	$x \times y$	$X \times Y$	p	P	溢出否
1	无符号乘	6	0110	10	1010	60	0011 1100	12	1100	溢出
2	带符号乘	6	0110	-6	1010	-36	1101 1100	-4	1100	溢出
3	无符号乘	8	1000	2	0010	16	0001 0000	0	0000	溢出
4	带符号乘	-8	1000	2	0010	-16	1111 0000	0	0000	溢出
5	无符号乘	13	1101	14	1110	182	1011 0110	6	0110	溢出
6	带符号乘	-3	1101	-2	1110	6	0000 0110	6	0110	不溢出
7	无符号乘	2	0010	12	1100	24	0001 1000	8	1000	溢出
8	带符号乘	2	0010	-4	1100	-8	1111 1000	-8	1000	不溢出

根据上述结论，带符号整数乘法运算可以采用无符号整数乘法器实现，只要最终取 $2n$ 位乘积中的低 n 位即可。送到无符号整数乘法器中的两个乘数 X 和 Y 是两个 0/1 序列，对于带符号整数 x 和 y 来说， X 和 Y 就是 x 和 y 的补码表示，对于无符号整数 x 和 y 来说， X 和 Y 就是 x 和 y 的无符号整数表示。不管在无符号整数乘法器中进行的是补码乘还是无符号数乘，在没有溢出的情况下，最终得到的 $2n$ 位乘积中的低 n 位总是正确的。因此，需要能够判断是否发生了溢出。

对于 n 位无符号整数 x 和 y 的乘法运算，结果只取低 n 位，相当于取模为 2^n 。若丢弃的高 n 位是非 0 数，则发生溢出。用公式表示如下，式中 p 是指取低 n 位乘积时对应的值。

$$p = \begin{cases} x \times y & (x \times y < 2^n) \quad \text{正常} \\ x \times y \bmod 2^n & (x \times y \geq 2^n) \quad \text{溢出} \end{cases}$$

如果无符号数乘法指令能够将高 n 位保存到一个寄存器中，则编译器可以根据该寄存器的内容采用相应的比较指令来进行溢出判断。例如，在 MIPS 处理器中，无符号数乘法指令 `multu` 会将两个 32 位无符号数相乘得到的 64 位乘积置于两个 32 位内部寄存器 Hi 和 Lo 中，因此，可以根据 Hi 寄存器是否为全 0 来进行溢出判断。有些指令系统中的乘法指令并不保留高 n 位，此时可根据两个乘数 x 、 y 与结果 $p = x \times y$ 的关系来判断。判断规则为：若满足 $x \neq 0$ 且 $p/x = y$ ，则没有发生溢出；否则溢出。例如，对于表 2.8 中序号为 7 的例子， $x=2$ ， $y=12$ ， $p=8$ ，显然 $8/2 \neq 12$ ，因此，发生了溢出。

对于 n 位带符号整数乘法运算，若采用上述所说的无符号整数乘法器实现，则得到的乘积高 n 位并不一定是乘积的补码表示的高 n 位，例如，对于表 2.8 中序号为 6 的例子，当 $x=-3$ ， $y=-2$ 时，在无符号整数乘法器中得到的 $2n$ 位乘积的机器数为 1011 0110，而不是真正的 $2n$ 位乘积的补码表示 0000 0110，因此，无法根据高 n 位为全 0 或全 1 来判断是否溢出。带符号整数乘法运算与无符号整数乘法运算一样，也可根据两个乘数 x 、 y 与结果 $p = x \times y$ 的关系来判断。判断规则为：若满足 $x \neq 0$ 且 $p/x = y$ ，则没有发生溢出；否则溢出。例如，对于表 2.8 中序号 8 的例子， $x=2$ ， $y=-4$ ， $p=-8$ ，显然 $-8/2 = -4$ ，因此，没有发生溢出。对于序号为 2 的例子， $x=6$ ， $y=-6$ ， $p=-4$ ，显然 $-4/6 \neq -6$ ，因此，发生了溢出。

有些处理器中会使用专门的补码乘法器来进行带符号整数乘法运算，一位补码乘法称为布斯 (Booth) 乘法，两位补码乘法称为改进的布斯乘法 (Modified Booth Algorithm, MBA)，也称为基 4 布斯乘法。采用专门的补码乘法器实现带符号整数运算得到的结果是 $2n$ 位乘积的补码表示。例如，对于表 2.8 中序号为 2 的例子， $x=6$ ， $y=-6$ ，若采用专门的补码乘法器，则得到的乘积的 $2n$ 位机器数为 1101 1100，而不是无符号整数乘法器的结果 0011 1100；对于表 2.8 中序号为 6 的例子，若采用专门的补码乘法器，则得到的乘积的 $2n$ 位机器数为 0000 0110，而不是 1011 0110。在这种采用专门的补码乘法运算的情况下，可以通过高 n 位和低 n 位之间的关系来进行溢出判断。判断规则是：若高 n 位中每一位都与低 n 位的最高位相同，则不溢出；否则溢出。例如，在 MIPS 处理器中，带符号整数乘法指令 `mult` 会将两个 32 位带符号整数相乘，得到的 64 位乘积置于两个 32 位内部寄存器 Hi 和 Lo 中，因此，可以根据 Hi 寄存器中的每一位是否等于 Lo 寄存器中的第一位来进行溢出判断。

通常乘法指令没有判断溢出的功能，如果程序本身不采用防止溢出的措施，而且编译器也

不生成相应的用于溢出处理的代码的话,就会发生一些由于整数溢出而带来的问题。因此,在现实世界中 $x^2 \geq 0$ 的结论在计算机世界并不一定成立。例如,在 C 语言中,若 x 和 y 为 `int` 型, $x = 65\ 535$, 则 $y = x * x = -131\ 071$, x 的机器数为 `0000FFFFH`, y 的机器数为 `FFFE0001H`。

例 2.31 以下程序段实现数组元素的复制,将一个具有 `count` 个元素的 `int` 型数组复制到堆中新申请的一块内存区域中,请说明该程序段存在什么漏洞,引起该漏洞的原因是什么。

```
1  /* 复制数组到堆中,count 为数组元素个数 */
2  int copy_array(int *array, int count) {
3      int i;
4      /* 在堆区申请一块内存 */
5      int *myarray = (int *) malloc(count*sizeof(int));
6      if (myarray == NULL)
7          return -1;
8      for (i = 0; i < count; i++)
9          myarray[i] = array[i];
10     return count;
11 }
```

解 该程序段存在整数溢出漏洞,当 `count` 的值很大时,第 5 行 `malloc` 函数的参数 `count * sizeof(int)` 会发生溢出,例如,在 32 位机器上实现时, `sizeof(int) = 4`,若参数 `count = 230 + 1`,因为 $(2^{30} + 1) \times 4 = 2^{32} + 4 \pmod{2^{32}} = 4$,因此 `malloc` 函数只会分配 4 个字节的空间,而在后面的 `for` 循环执行时,复制到堆中的数组元素实际上有 $(2^{32} + 4) = 4\ 294\ 967\ 300$ 个字节,远超过 4 个字节的空间,从而会破坏在堆中的其他数据,导致程序崩溃或行为异常,更可怕的是,如果攻击者利用这种漏洞,以引起整数溢出的参数来调用函数,通过数组复制过程把自己的程序置入内存中并启动执行,就会造成极大的安全问题。 ■

2002 年, Sun Microsystems 公司的 RPC XDR 库中所带的 `xdr_array()` 函数发生整数溢出漏洞,攻击者可以利用这个漏洞从远程或本地获取 `root` 权限。`xdr_array()` 函数中需要计算 `nodesize` 变量的值,它采用的方法可能会由于乘积太大而导致整数溢出,使得攻击者可以构造一个特殊的参数来触发整数溢出,以一段事先预设好的信息覆盖一个已经分配的堆缓冲区,造成远程服务崩溃或者改变内存数据并执行任意代码。由于很多厂商的操作系统都使用了 Sun 公司的 XDR 库或者基于 XDR 库进行开发,因此很多厂商的程序也受到此问题影响。

对于整数除法,因为商的绝对值不可能比被除数的绝对值更大,因而肯定不会发生溢出,也就不会像整数乘法运算那样发生漏洞。但是,在不能整除时需要进行舍入,通常按照朝 0 方向舍入,即正数商取比自身小的最接近整数,负数商取比自身大的最接近整数。除数不能为 0,否则会发生“异常”,需要调出操作系统中的异常处理程序来处理。

2.7.6 常量的乘除运算

由于整数乘法运算比移位和加法等运算所用时间长得多,通常一次乘法运算需要 10 个左右的时钟周期,而一次移位、加法和减法等运算只要一个或更少的时钟周期,因此,编译器在处理变量与常数相乘时,往往以移位、加法和减法的组合运算来代替乘法运算。例如,对于表达式 $x * 20$,编译器可以利用 $20 = 16 + 4 = 2^4 + 2^2$,将 $x * 20$ 转换为 $(x << 4) + (x << 2)$,这样,一次乘法转换成了两次移位和一次加法。不管是无符号整数还是带符号整数的乘法,即使

乘积溢出时，利用移位和加减运算组合的方式得到的结果都和采用直接相乘的结果是一样的。

对于整数除法运算，由于计算机中除法运算比较复杂，而且不能用流水线方式实现，所以一次除法运算大致需要 30 个或更多时钟周期。为了缩短除法运算的时间，编译器在处理一个变量与一个 2 的幂次形式的整数相除时，常采用右移运算来实现。无符号整数除法采用逻辑右移方式，带符号整数除法采用算术右移方式。两个整数相除，结果也一定是整数，在不能整除时，其商采用朝 0 方向舍入的方式，也就是截断方式，即将小数点后的数直接去掉，例如， $7/3=2$ ， $-7/3=-2$ 。

对于无符号整数来说，采用逻辑右移时，高位补 0，低位移出，因此，移位后得到的商的值只可能变小而不会变大，即商朝 0 方向舍入。因此，不管是否能够整除，采用移位方式和直接相除得到的商完全一样，如表 2.9 给出的例子所示。表 2.9 中给出了无符号整数 32 760 除以 2^k (k 为正整数) 的例子，无符号整数 32 760 的机器数为 0111 1111 1111 1000。

表 2.9 无符号整数 32 760 除以 2^k 的示例

k	32760 >> k		32760/ 2^k	
1	0 0111 1111 1111 100	16380	16380.0	16380
3	000 0111 1111 1111 1	2047	2047.0	2047
6	000000 0111 1111 11	511	511.875	511
8	00000000 0111 1111	127	127.96875	127

对于带符号整数来说，采用算术右移时，高位补符号，低位移出。因此，当符号为 0 时，与无符号整数相同，采用移位方式和直接相除得到的商完全一样。当符号为 1 时，若低位移出的是全 0，则说明能够整除，移位后得到的商与直接相除的完全一样；若低位移出的并非全 0，则说明不能整除，移出一个非 0 数相当于把商中小数点后面的值舍去。因为符号是 1，所以商是负数，一个补码表示的负数舍去小数部分的值后变得更小，因此移位后的结果是更小的负数商。例如，对于 $-3/2$ ，假定补码位数为 4，则进行算术右移操作 $1101 >> 1 = 1110.1B$ (小数点后面部分移出) 后得到的商为 -2 ，而精确商是 -1.5 (整数商应为 -1)。算术右移后得到的商比精确商少了 0.5，显然朝 $-\infty$ 方向进行了舍入，而不是朝 0 方向舍入。因此，这种情况下，移位得到的商与直接相除得到的商不一样，需要进行校正。校正的方法是，对于带符号整数 x ，若 $x < 0$ ，则在右移前，先将 x 加上偏移量 ($2^k - 1$)，然后再右移 k 位。例如，上述例子中，在对 -3 右移 1 位之前，先将 -3 加上 1，即先得到 $1101 + 0001 = 1110$ ，然后再算术右移，即 $1110 >> 1 = 1111$ ，此时商为 -1 。

表 2.10 给出了带符号整数 $-32\,760$ 除以 2^k (k 为正整数) 的例子，带符号整数 $-32\,760$ 的补码表示为 1000 0000 0000 1000。

表 2.10 带符号整数 $-32\,760$ 除以 2^k 的示例

k	偏移量	$-32760 + \text{偏移量}$	$(-32760 + \text{偏移量}) >> k$	$-32760/2^k$	
1	1	1000 0000 0000 1001	1 1000 0000 0000 100	-16380	-16380.0
3	7	1000 0000 0000 1111	111 1000 0000 0000 1	-2047	-2047.0
6	63	1000 0000 0100 0111	111111 1000 0000 01	-511	-511.875
8	255	1000 0001 0000 0111	11111111 1000 0001	-127	-127.96875

从表 2.10 可以看出, 对带符号整数 $-32\,760$ 先加一个偏移量后再进行算术右移, 避免了商朝 $-\infty$ 方向舍入的问题。例如, 对于表中 $k=6$ 的情况, 若不进行偏移校正, 则算术右移 6 位后商的补码表示为 111111 1000 0000 00, 即商为 -512 , 而校正后得到的商等于 $-32\,760/64$ 的整数商 -511 。

2.7.7 浮点数运算

浮点数不像整数那样有移位、扩展和截断等运算, 浮点数运算主要是加、减、乘和除运算。

1. 浮点数加减运算

先看一个十进制数加法运算的例子: $0.123 \times 10^5 + 0.456 \times 10^2$ 。显然, 不可以把 0.123 和 0.456 直接相加, 必须把阶调整为相等后才可实现两数相加。其计算过程如下。

$$\begin{aligned} 0.123 \times 10^5 + 0.456 \times 10^2 &= 0.123 \times 10^5 + 0.000\,456 \times 10^5 \\ &= (0.123 + 0.000\,456) \times 10^5 \\ &= 0.123\,456 \times 10^5 \end{aligned}$$

从上面的例子不难理解实现浮点数加减法的运算规则。

设两个规格化浮点数 x 和 y 表示为 $x = M_x \times 2^{E_x}$, $y = M_y \times 2^{E_y}$, M_x 、 M_y 分别是浮点数 x 和 y 的尾数, E_x 、 E_y 分别是浮点数 x 和 y 的阶, 不失一般性, 设 $E_x \leq E_y$, 那么

$$\begin{aligned} x + y &= (M_x \times 2^{E_x - E_y} + M_y) \times 2^{E_y} \\ x - y &= (M_x \times 2^{E_x - E_y} - M_y) \times 2^{E_y} \end{aligned}$$

计算机中实现上述计算过程需要经过对阶、尾数加减、规格化和舍入 4 个步骤, 此外, 还必须考虑溢出判断和溢出处理问题。

(1) 对阶

对阶的目的是使两数的阶相等, 以便尾数可以相加减。对阶的原则是: 小阶向大阶看齐, 阶小的那个数的尾数右移, 右移的位数等于两个阶的差的绝对值。通用计算机多采用 IEEE 754 标准来表示浮点数, 因此, 阶小的那个数的尾数右移时按原码小数方式右移, 符号位不参加移位, 数值位要将隐含的一位“1”右移到小数部分, 前面空出的位补 0。为了保证运算的精度, 尾数右移时, 低位移出的位不要丢掉, 应保留并参加尾数部分的运算。

(2) 尾数加减

对阶后两个浮点数的阶相等, 此时, 可以把对阶后的尾数相加减。因为 IEEE 754 采用定点原码小数表示尾数, 所以, 尾数加减实际上是定点原码小数的加减运算。在进行尾数加减时, 必须把隐藏位还原到尾数部分, 对阶过程中尾数右移时保留的附加位也要参加运算。

(3) 尾数规格化

IEEE 754 的规格化尾数形式为: $\pm 1.xx \cdots x$ 。在进行尾数相加减后可能会得到各种形式的结果, 例如:

$$\begin{aligned} 1.xx \cdots x + 1.xx \cdots x &= \pm 1x.xx \cdots x \\ 1.xx \cdots x - 1.xx \cdots x &= \pm 0.00 \cdots 01x \cdots x \end{aligned}$$

① 对于上述结果为 $\pm 1x.xx\cdots x$ 的情况, 需要进行如下右规操作: 尾数右移一位, 阶码加 1。最后一位移出时, 要考虑舍入。

② 对于上述结果为 $\pm 0.00\cdots 01x\cdots x$ 的情况, 需要进行如下左规操作: 数值位逐次左移, 阶码逐次减 1, 直到将第一位“1”移到小数点左边。尾数左移时数值部分最左 k 个 0 被移出, 因此, 相对来说, 小数点右移了 k 位。因为进行尾数相加时, 默认的小数点位置在第一个数值位 (即隐藏位) 之后, 所以小数点右移 k 位后被移到了第一位 1 后面, 这个 1 就是隐藏位。

(4) 尾数的舍入处理

在对阶和尾数右规时, 可能会对尾数进行右移, 为保证运算精度, 一般将低位移出的位保留下来, 并让其参与中间过程的运算, 最后再将运算结果进行舍入, 以还原表示成 IEEE 754 格式。这里要解决以下两个问题。

① 保留多少附加位才能保证运算的精度?

② 最终如何对保留的附加位进行舍入?

对于第①个问题, 可能无法给出一个准确的答案。但是不管怎么说, 保留附加位应该可以得到比不保留附加位更高的精度。IEEE 754 标准规定, 所有浮点数运算的中间结果右边都必须至少保留两位附加位。这两位附加位中, 紧跟在浮点数尾数右边那一位为保护位或警戒位 (guard), 用以保护尾数右移的位, 紧跟保护位右边的是舍入位 (round), 左规时可以根据其值进行舍入。在 IEEE 754 标准中, 为了更进一步提高计算精度, 在保护位和舍入位后面还引入了额外的一个数位, 称为粘位 (sticky)。只要舍入位的右边有任何非 0 数字, 粘位就被置 1; 否则, 粘位被置为 0。

对于第②个问题, IEEE 754 提供了 4 种可选模式: 就近舍入 (中间值舍入到偶数)、朝 $+\infty$ 方向舍入、朝 $-\infty$ 方向舍入、朝 0 方向舍入。

- 就近舍入到偶数。舍入为最近可表示的数, 当结果是两个可表示数的非中间值时, 实际上是“0 舍 1 入”方式。当结果正好在两个可表示数中间时, 根据“就近舍入”的原则无法操作。IEEE 754 标准规定: 这种情况下结果强迫为偶数。实现过程为: 若舍入后结果的 LSB 为 1 (即奇数), 则末位加 1; 否则舍入后的结果不变。这样, 就保证了结果的 LSB 总是 0 (即偶数)。

使用粘位可以减少运算结果正好在两个可表示数中间的情况。不失一般性, 我们用一个十进制数计算的例子来说明这样做的好处。假设计算 $1.24 \times 10^4 + 5.03 \times 10^1$ (精度保留两位小数), 若只使用保护位和舍入位而不使用粘位, 则结果为 $1.2400 \times 10^4 + 0.0050 \times 10^4 = 1.2450 \times 10^4$ 。这个结果位于两个相邻可表示数 1.24×10^4 和 1.25×10^4 的中间, 采用就近舍入到偶数, 则结果应该为 1.24×10^4 ; 若同时使用保护位、舍入位和粘位, 则结果为 $1.24000 \times 10^4 + 0.00503 \times 10^4 = 1.24503 \times 10^4$ 。这个结果就不在 1.24×10^4 和 1.25×10^4 的中间, 而更接近于 1.25×10^4 , 采用就近舍入方式, 结果应该为 1.25×10^4 。显然, 后者更精确。

- 朝 $+\infty$ 方向舍入。总是取数轴上右边最近可表示数, 也称为正向舍入或朝上舍入。
- 朝 $-\infty$ 方向舍入。总是取数轴上左边最近可表示数, 也称为负向舍入或朝下舍入。

- 朝0方向舍入。直接截取所需位数，丢弃后面所有位，也称为截取、截断或恒舍法。这种舍入处理最简单。对正数或负数来说，都是取数轴上更靠近原点的那个可表示数，是一种趋向原点的舍入，因此，又称为趋向0舍入。

表2.11以十进制小数为例给出了若干示例，以说明这4种舍入方式，表中假定结果保留小数点后面三位数，最后两位（加黑的数字）为附加位，需要舍去。

表2.11 以十进制小数为例对4种舍入方式举例

方式	2.05240	2.05250	2.05260	-2.05240	-2.05250	-2.05260
就近或偶数舍入	2.052	2.052	2.053	-2.052	-2.052	-2.053
朝+∞方向舍入	2.053	2.053	2.053	-2.052	-2.052	-2.052
朝-∞方向舍入	2.052	2.052	2.052	-2.053	-2.053	-2.053
朝0方向舍入	2.052	2.052	2.052	-2.052	-2.052	-2.052

(5) 阶码溢出判断

在进行尾数规格化和尾数舍入时，可能会对结果的阶码执行加、减运算。因此，必须考虑结果的阶码溢出问题。若结果的阶码为全1，也即，结果的阶比最大允许值127（单精度）或1023（双精度）还大，则发生“阶码上溢”，产生“阶码上溢”异常，也有的机器把结果置为“+∞”（符号位为0时）或“-∞”（符号位为1时），而不产生“溢出”异常。若结果的阶码为全0，也即，结果的阶比最小允许值（-126或-1022）还小，则发生“阶码下溢”，此时，一般把结果置为“+0”（符号位为0时）或“-0”（符号位为1时），也有的机器引起“阶码下溢”异常。从浮点数加、减运算过程可以看出，浮点数的溢出并不以尾数溢出来判断，尾数溢出可以通过右规操作得到纠正。因此结果是否溢出通过判断阶码是否上溢来确定。

2. 浮点数乘除运算

对于浮点数的乘除运算，在进行运算前首先应对参加运算的操作数进行判0处理、规格化操作和溢出判断，并确定参加运算的两个操作数是正常的规格化浮点数。

浮点数乘除运算步骤类似于浮点数加减运算步骤，两者主要区别是，加减运算需要对阶，而对乘除运算来说则无需这一步。两者对结果的后处理步骤一样，都包括规格化、舍入和阶码溢出处理。

已知两个浮点数 $x = M_x \times 2^{E_x}$, $y = M_y \times 2^{E_y}$ ，则乘、除运算的结果如下。

$$x \times y = (M_x \times 2^{E_x}) \times (M_y \times 2^{E_y}) = (M_x \times M_y) \times 2^{E_x + E_y}$$

$$x/y = (M_x \times 2^{E_x}) / (M_y \times 2^{E_y}) = (M_x/M_y) \times 2^{E_x - E_y}$$

3. 浮点运算时异常和精度等问题

计算机中的浮点数运算比较复杂，从浮点数的表示来说，有规格化浮点数和非规格化浮点数，有+∞、-∞和非数（NaN）等特殊数据的表示。利用这些特殊表示，程序可以实现诸如+∞+(-∞)、+∞-(+∞)、∞/∞、8.0/0等运算。

此外，由于浮点加减运算中需要对阶并最终进行舍入，因而可能导致“大数吃小数”的问题，使得浮点数运算不能满足加法结合律和乘法结合律。

例如，在 x 和 y 是单精度浮点类型时，当 $x = -1.5 \times 10^{30}$, $y = 1.5 \times 10^{30}$, $z = 1.0$ ，则：

$$(x + y) + z = (-1.5 \times 10^{30} + 1.5 \times 10^{30}) + 1.0 = 1.0$$

$$x + (y + z) = -1.5 \times 10^{30} + (1.5 \times 10^{30} + 1.0) = 0.0$$

根据上述计算可知, $(x + y) + z \neq x + (y + z)$, 其原因是, 当一个“大数”和一个“小数”相加时, 因为对阶使得“小数”尾数中的有效数字右移后被丢弃, 从而使“小数”变为 0。

例如, 在 x 和 y 是单精度浮点类型时, 当 $x = y = 1.0 \times 10^{30}$, $z = 1.0 \times 10^{-30}$, 则:

$$(x \times y) \times z = (1.0 \times 10^{30} \times 1.0 \times 10^{30}) \times 1.0 \times 10^{-30} = +\infty$$

$$x \times (y \times z) = 1.0 \times 10^{30} \times (1.0 \times 10^{30} \times 1.0 \times 10^{-30}) = 1.0 \times 10^{30}$$

显然, $(x \times y) \times z \neq x \times (y \times z)$, 这主要是两个大数相乘后可能超出可表示范围造成的。

1991 年 2 月 25 日, 海湾战争中, 美国在沙特阿拉伯达摩地区设置的爱国者导弹拦截伊拉克的飞毛腿导弹失败, 致使飞毛腿导弹击中了沙特阿拉伯载赫蓝的一个美军军营, 杀死了美国陆军第十四军需分队的 28 名士兵。这是由爱国者导弹系统时钟内的一个软件错误造成的, 引起这个软件错误的原因是浮点数的精度问题。爱国者导弹系统中有一个内置时钟, 用计数器实现, 每隔 0.1 秒计数一次。程序用 0.1 的一个 24 位定点二进制小数 x 来乘以计数值作为以秒为单位的时间。0.1 的二进制表示是一个无限循环序列: $0.00011[0011]\dots$, $x = 0.000\ 1100\ 1100\ 1100\ 1100\ 1100\ 1100\text{B}$ 。显然, x 只是 0.1 的近似表示, $0.1 - x = 0.000\ 1100\ 1100\ 1100\ 1100\ 1100[1100]\dots - 0.000\ 1100\ 1100\ 1100\ 1100\ 1100\ 1100\text{B}$, 即误差值为:

$$0.000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1100[1100]\dots\text{B} = 2^{-20} \times 0.1 \approx 9.54 \times 10^{-8}$$

在爱国者导弹准备拦截飞毛腿导弹之前, 已经连续工作了 100 小时, 相当于计数了 $100 \times 60 \times 60 \times 10 = 36 \times 10^5$ 次, 因而导弹的时钟已经偏差了 $9.54 \times 10^{-8} \times 36 \times 10^5 \approx 0.343$ 秒。

爱国者导弹根据飞毛腿导弹的速度乘以它被侦测到的时间来预测位置, 飞毛腿导弹的速度大约为 2000 米/秒, 因此, 由于系统时钟误差导致的距离误差相当于 $0.343 \times 2000 \approx 687$ 米。因此, 由于时钟误差, 纵使雷达系统侦察到飞毛腿导弹并且预计了它的弹道, 爱国者导弹却找不到实际上来袭的飞毛腿导弹。这种情况下, 起初的目标发现被视为一次假警报, 侦测到的目标也在系统中被删除。

实际上, 以色列方面已经发现了这个问题并于 1991 年 2 月 11 日知会了美国陆军及爱国者计划办公室 (软件制造商)。以色列方面建议重新启动爱国者系统的电脑作为暂时解决方案, 可是美国陆军方面却不知道每次需要间隔多少时间重新启动系统一次。1991 年 2 月 16 日, 制造商向美国陆军提供了更新软件, 但这个软件最终却在飞毛腿导弹击中军营后的一天才运抵部队。

例 2.32 对于上述爱国者导弹拦截飞毛腿导弹的例子, 回答下列问题。

① 如果用精度更高一点的 24 位定点小数 $x = 0.000\ 1100\ 1100\ 1100\ 1100\ 1101\text{B}$ 来表示 0.1, 则 0.1 与 x 的偏差是多少? 系统运行 100 小时后的时钟偏差是多少? 在飞毛腿导弹速度为 2000 米/秒的情况下, 预测的距离偏差为多少?

② 假定用一个类型为 float 的变量 x 来表示 0.1, 则变量 x 在机器中的机器数是什么 (要求写成十六进制形式)? 0.1 与 x 的偏差是多少? 系统运行 100 小时后的时钟偏差是多少? 在飞毛腿导弹速度为 2000 米/秒的情况下, 预测的距离偏差为多少?

③ 如果将 0.1 用 32 位二进制定点小数 $x = 0.000\ 1100\ 1100\ 1100\ 1100\ 1100\ 1101\ \text{B}$ 表示, 则其误差比用 32 位 float 表示的误差更大还是更小? 试分析这两种方案的优缺点。

解 ① 0.1 与 x 的偏差计算如下:

$$\begin{aligned} & |0.000\ 1100\ 1100\ 1100\ 1100\ 1100[1100]\cdots - 0.000\ 1100\ 1100\ 1100\ 1101\ \text{B}| \\ &= 0.000\ 0000\ 0000\ 0000\ 0000\ 0000\ 00\ 1100[1100]\cdots \text{B} = 2^{-22} \times 0.1 \approx 2.38 \times 10^{-8} \end{aligned}$$

100 小时后的时钟偏差是 $2.38 \times 10^{-8} \times 36 \times 10^5 \approx 0.086$ 秒。预测的距离偏差为 $0.086 \times 2000 \approx 171$ 米。比爱国者导弹系统精确约 4 倍。

② $0.1 = 0.0\ 0011[0011]\ \text{B} = +1.1\ 0011\ 0011\ 0011\ 0011\ 0011\ 00\text{B} \times 2^{-4}$, float 类型采用 IEEE 754 单精度浮点数格式。符号位 s 为 0, 阶码 $e = 127 - 4 = 0111\ 1011\ \text{B}$, 尾数的小数部分为 $0.100\ 1100\ 1100\ 1100\ 1100\ 1100$, 因此, 在机器中 float 型变量 x 表示为 $0\ 011\ 1101\ 1\ 100\ 1100\ 1100\ 1100\ 1100$, 用十六进制形式表示为 3DCCCCCH。

由于 float 类型的精度有限, 只有 24 位有效位数, 尾数从最前面的 1 开始一共只能表示 24 位, 后面的有效数字全部被截断, 故 x 与 0.1 之间的误差为: $|x - 0.1| = 0.000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1100[1100]\cdots \text{B}$ 。这个值等于 $2^{-24} \times 0.1$, 大约为 5.96×10^{-9} 。100 小时后的时钟偏差是 $5.96 \times 10^{-9} \times 36 \times 10^5 \approx 0.0215$ 秒。预测的距离偏差仅为 $0.0215 \times 2000 \approx 43$ 米。比爱国者导弹系统精确约 16 倍。

③ 当 $x = 0.000\ 1100\ 1100\ 1100\ 1100\ 1100\ 1101\ \text{B}$ 时, 与 0.1 之间的误差为: $|x - 0.1| = 0.000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 00\ 1100[1100]\cdots \text{B}$ 。这个值等于 $2^{-30} \times 0.1$, 大约为 9.31×10^{-11} 。100 小时后的时钟偏差是 $9.31 \times 10^{-11} \times 36 \times 10^5 \approx 0.000\ 335$ 秒。预测的距离偏差仅为 $0.000\ 335 \times 2000 \approx 0.67$ 米。比爱国者导弹系统精确约 1024 倍。 ■

从上述结果可以看出, 如果爱国者导弹系统中的 0.1 采用 32 位二进制定点小数表示, 那么将比采用 32 位 IEEE 754 浮点数标准 (float) 精度更高, 精确度大约高 $2^6 = 64$ 倍。而且, 采用 float 表示在计算速度上也会有很大影响, 因为必须先把计数值转换为 IEEE 754 格式浮点数, 然后再对两个 IEEE 754 格式的数进行相乘, 显然比直接将两个二进制数相乘要慢得多。

从上面这个例子可以看出, 程序员在编写程序时, 必须对底层机器级数据的表示和运算有深刻的理解, 而且在计算机世界里, 经常是“差之毫厘, 失之千里”, 需要细心再细心, 精确再精确。同时, 也不能遇到小数就用浮点数表示, 有些情况下, 例如需要将一个整数变量乘以一个确定的小数常量时, 可以先用一个确定的定点整数与整数变量相乘, 然后再通过移位运算来确定小数点。

2.8 小结

对指令来说数据就是一串 0/1 序列。根据指令的类型, 对应的 0/1 序列可能看成是一个无符号整数或带符号整数或浮点数或位串 (即非数值数据, 如逻辑值或 ASCII 码或汉字内码等)。无符号整数是正整数, 用来表示地址等; 带符号整数用补码表示; 浮点数表示实数, 大多用 IEEE 754 标准表示。有的指令系统还提供一种能处理 BCD 码的指令, BCD 码指二进制表示的十进制数, 一般用 8421 码表示。

对于计算机硬件来说,数据是没有类型的,所有数据就是一串 0/1 序列,称为机器数,机器数被送到特定的电路,按照指令规定的动作在计算机中进行运算、存储和传送。因此,机器数只能写成二进制形式,为了简化书写,在屏幕或纸上通常将二进制形式缩写成十六进制形式。

数据的宽度通常以字节(Byte)为基本单位表示,数据长度单位(如 MB、GB、TB 等)在表示容量和带宽等不同量时所代表的大小不同。数据的排列有大端和小端两种方式。大端方式以 MSB 所在地址为数据的地址,即给定地址处存放的是数据最高有效字节;小端方式以 LSB 所在地址为数据的地址,即给定地址处存放的是数据最低有效字节。

对于数据的运算,在用高级语言编程时需要注意带符号整数和无符号整数之间的转换问题。例如,C 语言支持隐式强制类型转换,因而,可能会因为强制类型转换而出现一些意想不到的问题,并导致程序运行的结果出错。此外,计算机中运算部件位数有限,导致计算机中算术运算的结果可能发生溢出,因而,在某些情况下,计算机世界里的算术运算不同于日常生活中的算术运算,不能想当然地用日常生活中算术运算的性质来判断计算机世界中的算术运算结果。例如,计算机世界中浮点运算不支持结合律,但可以给负数开根号等。

习题

1. 给出以下概念的解释说明。

真值	机器数	数值数据	非数值数据	无符号整数	带符号整数
定点数	原码	补码	变形补码	溢出	浮点数
尾数	阶	阶码	移码	阶码下溢	阶码上溢
规格化数	左规	右规	非规格化数	机器零	非数(NaN)
BCD 码	逻辑数	ASCII 码	汉字输入码	汉字内码	机器字长
大端方式	小端方式	最高有效位	最高有效字节(MSB)		最低有效位
最低有效字节(LSB)	掩码	算术移位	逻辑移位		0 扩展
符号扩展	零标志 ZF	溢出标志 OF	符号标志 SF	进位/借位标志 CF	

2. 简单回答下列问题。

- (1) 为什么计算机内部采用二进制表示信息?既然计算机内部所有信息都用二进制表示,为什么还要用到十六进制或八进制数?
- (2) 常用的定点数编码方式有哪几种?通常它们各自用来表示什么?
- (3) 为什么现代计算机中大多用补码表示带符号整数?
- (4) 在浮点数的基数和总位数一定的情况下,浮点数的表示范围和精度分别由什么决定?两者如何相互制约?
- (5) 为什么要对浮点数进行规格化?有哪两种规格化操作?
- (6) 为什么有些计算机中除了用二进制外还用 BCD 码来表示数值数据?
- (7) 为什么计算机处理汉字时会涉及不同的编码(如,输入码、内码、字模码)?说明这些编码中哪些用二进制编码,哪些不用二进制编码,为什么?

3. 实现下列各数的转换。

$$(1) (25.8125)_{10} = (?)_2 = (?)_8 = (?)_{16}$$

$$(2) (101101.011)_2 = (?)_{10} = (?)_8 = (?)_{16} = (?)_{8421}$$

$$(3) (0101\ 1001\ 0110.0011)_{8421} = (?)_{10} = (?)_2 = (?)_{16}$$

$$(4) (4E.C)_{16} = (?)_{10} = (?)_2$$

4. 假定机器数为8位(1位符号,7位数值),写出下列各二进制数的原码表示。

$$+0.1001, -0.1001, +1.0, -1.0, +0.010100, -0.010100, +0, -0$$

5. 假定机器数为8位(1位符号,7位数值),写出下列各二进制数的补码和移码表示。

$$+1001, -1001, +1, -1, +10100, -10100, +0, -0$$

6. 已知 $[x]_{\text{补}}$, 求 x 。

$$(1) [x]_{\text{补}} = 11100111 \quad (2) [x]_{\text{补}} = 10000000$$

$$(3) [x]_{\text{补}} = 01010010 \quad (4) [x]_{\text{补}} = 11010011$$

7. 某32位字长的机器中带符号整数用补码表示,浮点数用IEEE 754标准表示,寄存器R1和R2的内容分别为R1: 00 00 10 8BH, R2: 80 80 10 8BH。不同指令对寄存器进行不同的操作,因而不同指令执行时寄存器内容对应的真值不同。假定执行下列运算指令时,操作数为寄存器R1和R2的内容,则R1和R2中操作数的真值分别为多少?

(1) 无符号整数加法指令。

(2) 带符号整数乘法指令。

(3) 单精度浮点数减法指令。

8. 假定机器M的字长为32位,用补码表示带符号整数。下表中第一列给出了在机器M上执行的C语言程序中的关系表达式,请参照已有的表栏内容完成表中后三栏内容的填写。

关系表达式	运算类型	结果	说明
0 == 0U	无符号整数 有符号整数	0	11...1B ($2^{32} - 1$) > 00...0B (0) 011...1B ($2^{31} - 1$) > 100...0B (-2^{31})
-1 < 0			
-1 < 0U			
2147483647 > -2147483647 - 1			
2147483647U > -2147483647 - 1			
2147483647 > (int) 2147483648U			
-1 > -2			
(unsigned) -1 > -2			

9. 在32位计算机中运行一个C语言程序,在该程序中出现了以下变量的初值,请写出它们对应的机器数(用十六进制表示)。

$$(1) \text{int } x = -32768 \quad (2) \text{short } y = 522 \quad (3) \text{unsigned } z = 65530$$

$$(4) \text{char } c = '\text{@}' \quad (5) \text{float } a = -1.1 \quad (6) \text{double } b = 10.5$$

10. 在32位计算机中运行一个C语言程序,在该程序中出现了变量,已知这些变量在某一时刻的机器数(用十六进制表示)如下,请写出它们对应的真值。

$$(1) \text{int } x: \text{FFFF0006H} \quad (2) \text{short } y: \text{DFFCH} \quad (3) \text{unsigned } z: \text{FFFFFFFAH}$$

$$(4) \text{char } c: 2\text{AH} \quad (5) \text{float } a: \text{C4480000H} \quad (6) \text{double } b: \text{C024800000000000H}$$

11. 以下给出的是一些字符串变量的机器码, 请根据 ASCII 码定义写出对应的字符串。

(1) `char *mystring1:68H 65H 6CH 6CH 6FH 2CH 77H 6FH 72H 6CH 64H 0AH 00H`

(2) `char *mystring2:77H 65H 20H 61H 72H 65H 20H 68H 61H 70H 70H 79H 21H 00H`

12. 以下给出的是一些字符串变量的初值, 请写出对应的机器码。

(1) `char *mystring1="./myfile"` (2) `char *mystring2="OK, good!"`

13. 已知 C 语言中的按位异或运算 (XOR) 用符号 “^” 表示。对于任意一个位序列 a , $a^a=0$, C 语言程序可以利用这个特性来实现两个数值交换的功能。以下是一个实现该功能的 C 语言函数:

```
1 void xor_swap(int *x, int *y)
2 {
3     *y=*x ^ *y; /* 第一步 */
4     *x=*x ^ *y; /* 第二步 */
5     *y=*x ^ *y; /* 第三步 */
6 }
```

假定执行该函数时 $*x$ 和 $*y$ 的初始值分别为 a 和 b , 即 $*x=a$ 且 $*y=b$, 请给出每一步执行结束后, x 和 y 各自指向的内存单元中的内容分别是什么?

14. 假定某个实现数组元素倒置的函数 `reverse_array` 调用了第 13 题中给出的 `xor_swap` 函数:

```
1 void reverse_array(int a[], int len)
2 {
3     int left, right=len-1;
4     for (left=0; left<=right; left++, right--)
5         xor_swap(&a[left], &a[right]);
6 }
```

当 len 为偶数时, `reverse_array` 函数的执行没有问题。但是, 当 len 为奇数时, 函数的执行结果不正确。请问, 当 len 为奇数时会出现什么问题? 最后一次循环中的 `left` 和 `right` 各取什么值? 最后一次循环中调用 `xor_swap` 函数后的返回值是什么? 对 `reverse_array` 函数做怎样的改动就可消除该问题?

15. 假设以下表中的 x 和 y 是某 C 语言程序中的 `char` 型变量, 请根据 C 语言中的按位运算和逻辑运算的定义, 填写下表, 要求用十六进制形式填写。

x	y	x^y	$x\&y$	$x y$	$\sim x \sim y$	$x\&!y$	$x\&\&y$	$x y$	$!x !y$	$x\&\&\sim y$
0x5F	0xA0									
0xC7	0xF0									
0x80	0x7F									
0x07	0x55									

16. 对于一个 $n(n \geq 8)$ 位的变量 x , 写出满足下列要求的 C 语言表达式。

(1) x 的最高有效字节不变, 其余各位全变为 0。

(2) x 的最低有效字节不变, 其余各位全变为 0。

(3) x 的最低有效字节全变为 0, 其余各位取反。

(4) x 的最低有效字节全变为 1, 其余各位不变。

17. 以下是由反汇编器生成的一段针对某个小端方式处理器的机器级代码表示文本, 其中, 左边是指令所在的存储单元地址, 冒号后面是指令的机器码, 最右边是指令的汇编语言表

示,即汇编指令。已知反汇编输出中的机器数都采用补码表示,请给出指令代码中画线部分表示的机器数对应的真值。

```

80483d2: 81 ec b8 01 00 00    sub    &0x1b8, %esp
80483d8: 8b 55 08                mov    0x8(%ebp), %edx
80483db: 83 c2 14              add    $0x14, %edx
80483de: 8b 85 58 fe ff ff      mov    0xfffffe58(%ebp), %eax
80483e4: 03 02                  add    (%edx), %eax
80483e6: 89 85 74 fe ff ff      mov    %eax, 0xfffffe74(%ebp)
80483ec: 8b 55 08                mov    0x8(%ebp), %edx
80483ef: 83 c2 44              add    $0x44, %edx
80483f2: 8b 85 c8 fe ff ff      mov    0xfffffec8(%ebp), %eax
80483f8: 89 02                  mov    %eax, (%edx)
80483fa: 8b 45 10              mov    0x10(%ebp), %eax
80483fd: 03 45 0c              add    0xc(%ebp), %eax
8048400: 89 85 ec fe ff ff      mov    %eax, 0xfffffec(%ebp)
8048406: 8b 45 08                mov    0x8(%ebp), %eax
8048409: 83 c0 20              add    $0x20, %eax

```

18. 假设以下 C 语言函数 `compare_str_len` 用来判断两个字符串的长度,当字符串 `str1` 的长度大于 `str2` 的长度时函数返回值为 1,否则为 0。

```

1 int compare_str_len(char *str1, char *str2)
2 {
3     return strlen(str1) - strlen(str2) > 0;
4 }

```

已知 C 语言标准库函数 `strlen` 原型声明为“`size_t strlen(const char *s);`”,其中, `size_t` 被定义为 `unsigned int` 类型。请问:函数 `compare_str_len` 在什么情况下返回的结果不正确?为什么?为使函数正确返回结果应如何修改代码?

19. 考虑以下 C 语言程序代码:

```

1 int func1(unsigned word)
2 {
3     return (int)((word <<24) >> 24);
4 }
5 int func2(unsigned word)
6 {
7     return ((int)word <<24) >> 24;
8 }

```

假设在一个 32 位机器上执行这些函数,该机器使用二进制补码表示带符号整数。无符号数采用逻辑移位,带符号整数采用算术移位。请填写下表,并说明函数 `func1` 和 `func2` 的功能。

w		$\text{func1}(w)$		$\text{func2}(w)$	
机器数	值	机器数	值	机器数	值
	127				
	128				
	255				
	256				

20. 填写下表,注意对比无符号整数和带符号整数的乘法结果,以及截断操作前、后的结果。

模式	x		y		x × y (截断前)		x × y (截断后)	
	机器数	值	机器数	值	机器数	值	机器数	值
无符号	110		010					
带符号	110		010					
无符号	001		111					
带符号	001		111					
无符号	111		111					
带符号	111		111					

21. 以下是两段 C 语言代码，函数 arith() 是直接 C 语言写的，而 optarith() 是对 arith() 函数以某个确定的 M 和 N 编译生成的机器代码反编译生成的。根据 optarith()，可以推断函数 arith() 中 M 和 N 的值各是多少？

```
#define M
#define N
int arith (int x, int y)
{
    int result = 0 ;
    result = x* M + y/N;
    return result;
}

int optarith (int x, int y)
{
    int t = x;
    x <<= 4;
    x - = t;
    if (y < 0 ) y += 3;
    y >> 2;
    return x + y;
}
```

22. 下列几种情况所能表示的数的范围是什么？

- (1) 16 位无符号整数。
- (2) 16 位原码定点小数。
- (3) 16 位移码定点整数。
- (4) 16 位补码定点整数。
- (5) 下述格式的浮点数（基数为 2，移码的偏置常数为 128）。

符号	阶码	尾数
1 位	8 位移码	7 位原码数值部分

23. 以 IEEE 754 单精度浮点数格式表示下列十进制数。

+1.75, +19, -1/8, 258

24. 设一个变量的值为 4098，要求分别用 32 位补码整数和 IEEE 754 单精度浮点格式表示该变量（结果用十六进制形式表示），并说明哪段二进制位序列在两种表示中完全相同，为什么会相同？

25. 设一个变量的值为 -2 147 483 647，要求分别用 32 位补码整数和 IEEE754 单精度浮点格式

表示该变量（结果用十六进制形式表示），并说明哪种表示其值完全精确，哪种表示的是近似值（提示： $2\,147\,483\,647 = 2^{31} - 1$ ）。

26. 下表给出了有关 IEEE 754 浮点格式表示中一些重要的非负数的取值，表中已经有最大规格化数的相应内容，要求填入其他浮点数字格式的相应内容。

项目	阶码	尾数	单精度		双精度	
			以 2 的幂次表示的值	以 10 的幂次表示的值	以 2 的幂次表示的值	以 10 的幂次表示的值
0 1 最大规格化数 最小规格化数 最大非规格化数 最小非规格化数 + ∞ NaN	11111110	1...11	$(2 - 2^{-23}) \times 2^{127}$	3.4×10^{38}	$(2 - 2^{-52}) \times 2^{1023}$	1.8×10^{308}

27. 已知下列字符编码：A 为 100 0001，a 为 110 0001，0 为 011 0000，求 E、e、f、7、G、Z、5 的 7 位 ACSII 码和在第一位前加入奇校验位后的 8 位编码。
28. 假定在一个程序中定义了变量 x 、 y 和 i ，其中， x 和 y 是 float 型变量（用 IEEE754 单精度浮点数表示）， i 是 16 位 short 型变量（用补码表示）。程序执行到某一时刻， $x = -0.125$ 、 $y = 7.5$ 、 $i = 100$ ，它们都被写到了主存（按字节编址），其地址分别是 100、108 和 112。请分别画出在大端模式机器和小端模式机器上变量 x 、 y 和 i 中每个字节在主存的存放位置。
29. 对于图 2.6，假设 $n = 8$ ，机器数 X 和 Y 的真值分别是 x 和 y 。请按照图 2.6 的功能填写下表并给出解释。要求机器数用十六进制形式填写，真值用十进制形式填写。

表示	X	x	Y	y	$X + Y$	$x + y$	OF	SF	CF	$X - Y$	$x - y$	OF	SF	CF
无符号	0xB0		0x8C											
带符号	0xB0		0x8C											
无符号	0x7E		0x5D											
带符号	0x7E		0x5D											

30. 某 32 位计算机上，有一个函数其原型声明为 “int ch_mul_overflow(int x, int y);”，该函数用于对两个 int 型变量 x 和 y 的乘积判断是否溢出，若溢出则返回 1，否则返回 0。请使用 64 位精度的整数类型 long long 来编写该函数。
31. 对于 2.7.5 节中例 2.31 存在的整数溢出漏洞，如果将其中的第 5 行改为以下两个语句：
`unsigned long long arraysize = count*(unsigned long long)sizeof(int);`
`int *myarray = (int *) malloc(arraysize);`
已知 C 语言标准库函数 malloc 的原型声明为 “void * malloc (size_t size);”，其中，size_t 定义为 unsigned int 类型，则上述改动能否消除整数溢出漏洞？若能则说明理由；若不能则给出修改方案。
32. 已知一次整数加法、一次整数减法和一次移位操作都只需一个时钟周期，一次整数乘法操作需要 10 个时钟周期。若 x 为一个整型变量，现要计算 $55 * x$ ，请给出一种计算表达式，

使得所用时钟周期数最少。

33. 假设 x 为一个 int 型变量, 请给出一个用来计算 $x/32$ 的值的函数 `div32`。要求不能使用除法、乘法、模运算、比较运算、循环语句和条件语句, 可以使用右移、加法以及任何按位运算。
34. 无符号整数变量 ux 和 uy 的声明和初始化如下:

```
unsigned ux = x;
unsigned uy = y;
```

若 `sizeof(int) = 4`, 则对于任意 int 型变量 x 和 y , 判断以下关系表达式是否永真。若永真则给出证明; 若不永真则给出结果为假时 x 和 y 的取值。

- | | |
|---|---------------------------------------|
| (1) $(x*x) >= 0$ | (2) $(x-1 < 0) \parallel x > 0$ |
| (3) $x < 0 \parallel -x <= 0$ | (4) $x > 0 \parallel -x >= 0$ |
| (5) $x \& 0xf != 15 \parallel (x < 28) < 0$ | (6) $x > y == (-x < -y)$ |
| (7) $\sim x + \sim y == \sim (x + y)$ | (8) $(int)(ux - uy) == -(y - x)$ |
| (9) $((x >> 2) < 2) <= x$ | (10) $x*4 + y*8 == (x < 2) + (y < 3)$ |
| (11) $x/4 + y/8 == (x >> 2) + (y >> 3)$ | (12) $x*y == ux*uy$ |
| (13) $x + y == ux + uy$ | (14) $x* \sim y + ux*uy == -x$ |

35. 变量 dx 、 dy 和 dz 的声明和初始化如下:

```
double dx = (double) x;
double dy = (double) y;
double dz = (double) z;
```

若 float 和 double 分别采用 IEEE 754 单精度和双精度浮点数格式, `sizeof(int) = 4`, 则对于任意 int 型变量 x 、 y 和 z , 判断以下关系表达式是否永真。若永真则给出证明; 若不永真则给出结果为假时 x 、 y 和 z 的取值。

- | | |
|----------------------------------|--|
| (1) $dx*dx >= 0$ | (2) $(double)(float) x == dx$ |
| (3) $dx + dy == (double)(x + y)$ | (4) $(dx + dy) + dz == dx + (dy + dz)$ |
| (5) $dx*dy*dz == dz*dy*dx$ | (6) $dx/dx == dy/dy$ |

36. 在 IEEE 754 浮点数运算中, 当结果的尾数出现什么形式时需要进行左规, 什么形式时需要进行右规? 如何进行左规, 如何进行右规?
37. 在 IEEE 754 浮点数运算中, 如何判断浮点运算的结果是否溢出?
38. 分别给出不能精确用 IEEE 754 单精度和双精度格式表示的最小正整数。
39. 采用 IEEE 754 单精度浮点数格式计算下列表达式的值。

- | | |
|-----------------------|-----------------------|
| (1) $0.75 + (-65.25)$ | (2) $0.75 - (-65.25)$ |
|-----------------------|-----------------------|

40. 以下是函数 `fpower2` 的 C 语言源程序, 它用于计算 2^x 的浮点数表示, 其中调用了函数 `u2f`, `u2f` 用于将一个无符号整数表示的 0/1 序列作为 float 类型返回。请填写 `fpower2` 函数中的空白部分, 以使其能正确计算结果。

```
1 float fpower2(int x)
2 {
3     unsigned exp, frac, u;
4
5     if (x < _____) { /* 值太小, 返回 0.0 */
```

```

6      exp = _____;
7      frac = _____;
8  } else if (x < _____) { /* 返回非规格化结果 */
9      exp = _____;
10     frac = _____;
11 } else if (x < _____) { /* 返回规格化结果 */
12     exp = _____;
13     frac = _____;
14 } else { /* 值太大, 返回 +∞ */
15     exp = _____;
16     frac = _____;
17 }
18 u = exp << 23 | frac;
19 return u2f(u);
20 }

```

41. 以下是一组关于浮点数按位级进行运算的编程题目，其中用到一个数据类型 `float_bits`，它被定义为 `unsigned int` 类型。以下程序代码必须采用 IEEE 754 标准规定的运算规则，例如，舍入应采用就近舍入到偶数的方式。此外，代码中不能使用任何浮点数类型、浮点数运算和浮点常数，只能使用 `float_bits` 类型；不能使用任何复合数据类型，如数组、结构和联合等；可以使用无符号整数或带符号整数的数据类型、常数和运算。要求编程实现以下功能并进行正确性测试。

(1) 计算浮点数 f 的绝对值 $|f|$ 。若 f 为 NaN，则返回 f ，否则返回 $|f|$ 。函数原型为：

```
float_bits float_abs(float_bits f);
```

(2) 计算浮点数 f 的负数 $-f$ 。若 f 为 NaN，则返回 f ，否则返回 $-f$ 。函数原型为：

```
float_bits float_neg(float_bits f);
```

(3) 计算 $0.5 * f$ 。若 f 为 NaN，则返回 f ，否则返回 $0.5 * f$ 。函数原型为：

```
float_bits float_half(float_bits f);
```

(4) 计算 $2.0 * f$ 。若 f 为 NaN，则返回 f ，否则返回 $2.0 * f$ 。函数原型为：

```
float_bits float_twice(float_bits f);
```

(5) 将 `int` 型整数 i 的位序列转换为 `float` 型位序列。函数原型为：

```
float_bits float_i2f(int i);
```

(6) 将浮点数 f 的位序列转换为 `int` 型位序列。若 f 为非规格化数，则返回值为 0；若 f 是 NaN 或 $\pm \infty$ 或超出 `int` 型数可表示范围，则返回值为 `0x80000000`；若 f 带小数部分，则考虑舍入。函数原型为：

```
int float_f2i(float_bits f);
```